Electrónica, Control y Automatización con ESP32

MECRO · 8 semanas · 16 sesiones · ESP32 clásico

Cuaderno de estudio para jóvenes universitarios y entusiastas. La ruta integra los veinte proyectos del laboratorio MECRO y los fundamentos de programación con una progresión de medición, actuación y control. Las aplicaciones a drones se trabajan mediante subsistemas y pruebas en tierra.

Organización recomendada: catorce clases en línea de dos horas y dos talleres presenciales de cuatro horas, clases 8 y 16. Total: treinta y seis horas de contacto. Se puede estudiar con simulación y usar hardware compartido en taller. El kit opcional debe contrastarse con el inventario confirmado.

1. Electrónica con ESP32: del circuito al primer programa
2. Programación y entradas digitales: botones que significan eventos
3. Temporización y máquinas de estados: atender más de una tarea
4. ADC y calibración: de una lectura a una decisión
5. PWM, sonido y salidas digitales en serie
6. Sensores digitales, I2C y pantallas: un dato con identidad
7. Distancia: tiempo de vuelo, adaptación de nivel y lecturas inválidas
8. Taller 1: servomotor y explorador ultrasónico
9. Motores y etapa de potencia: la orden no es la energía
10. Encoder e interrupciones: medir velocidad con una definición
11. Control realimentado: P, PI y una introducción a PID
12. IMU y orientación de banco: medir antes de estabilizar
13. Mando y control de banco: interpretar el proyecto de vuelo
14. Datos en tierra: Wi-Fi, HTTP y marcas de tiempo
15. Proyecto integrador: una estación de pruebas para UAV
16. Taller final: pruebas, diagnóstico y clase maestra de cierre

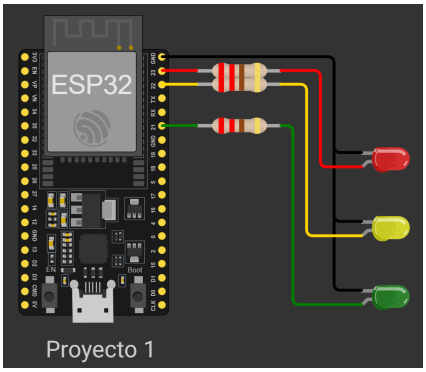
Los casos numéricos y los datos sintéticos sirven para aprender y no se presentan como mediciones realizadas por MECRO. Las imágenes del laboratorio muestran referencias de simulación; cada lectura explica las revisiones necesarias para hardware físico. El software nuevo se compiló para la versión fijada; las verificaciones de banco se realizan antes y durante la docencia.

Qué estamos construyendo

Clase 01 · Electrónica con ESP32: del circuito al primer programa · 1/6

Al terminar Reconocer la placa clásica y construir una salida LED cuya corriente pueda explicar.

El primer objeto del curso es un LED externo con su resistencia. Parece sencillo, pero reúne las cuatro decisiones que repetiremos: de dónde sale la energía, por dónde circula la corriente, qué señal controla el comportamiento y cómo comprobaremos el resultado. El ESP32 es un microcontrolador: ejecuta instrucciones y dispone de periféricos para relacionarlas con entradas y salidas. La placa de desarrollo añade alimentación, conexiones y una interfaz de programación. El módulo metálico y el conector USB cumplen funciones diferentes.



MECRO, proyecto 01. Esquema original del laboratorio para reconocer la arquitectura en Wokwi. Para montaje físico se aplica la tabla y las correcciones de esta lectura.

Identifica el texto del módulo antes de elegir la placa en el entorno. El curso usa ESP32 clásico, confirmado para el kit MECRO. Una placa S3 requiere otra selección y otro mapa de pines. En el protoboard, los grupos de contactos y los rieles de alimentación no son todos continuos. Con el circuito desconectado, sigue con el dedo el camino GPIO, resistencia, LED y GND. Si el camino termina en una fila sin continuidad, ningún cambio de código lo completará.

La referencia GND permite interpretar el voltaje de una señal. Medir 3,3 V significa medir una diferencia respecto a esa referencia; no significa que el pin pueda entregar cualquier corriente. Un GPIO representa una orden de baja potencia. Durante las primeras prácticas separaremos esa función de la energía que necesitan servos y motores. Esta distinción será fundamental al reconocer el controlador, los ESC y la batería de un dron.

Voltaje, corriente y resistencia con unidades

Clase 01 · Electrónica con ESP32: del circuito al primer programa · 2/6

El voltaje expresa diferencia de potencial y se mide en voltios. La corriente expresa flujo de carga y se mide en amperios. La resistencia relaciona ambos en un elemento resistivo bajo el modelo que usamos en clase. La analogía del agua ayuda a imaginar presión y caudal, pero el circuito se resuelve con magnitudes eléctricas. La corriente se expresa con frecuencia en miliamperios: 5 mA son 0,005 A. Convertir correctamente la unidad evita seleccionar una resistencia mil veces distinta.

$$V_R = IR, \qquad P_R = V_R I = I^2 R$$

La ley de Ohm se aplica a la resistencia. El LED tiene una caída de tensión propia; no es una resistencia ideal.

Magnitudes que vamos a distinguir

Magnitud	Unidad	Qué pregunta responde
Voltaje	V	Diferencia entre dos nodos
Corriente	A o mA	Cuánta carga circula
Resistencia	ohm	Relación V/I del resistor
Potencia	W	Energía por segundo

El LED tiene polaridad y una curva corriente-tensión. Para el cálculo inicial tomamos una caída de tensión aproximada del componente y dejamos el resto a la resistencia. Esa aproximación debe declararse: un LED rojo y uno azul no tienen necesariamente la misma caída. El valor que medimos en el GPIO también puede diferir del ideal cuando entrega corriente. El cálculo sirve para proponer el circuito y la medición sirve para contrastar la propuesta.

Un cálculo que decide una pieza

Clase 01 · Electrónica con ESP32: del circuito al primer programa · 3/6

LED rojo del ejercicio

Supón GPIO de 3,3 V, LED con caída aproximada de 2,0 V y objetivo de 5 mA. ¿Qué resistencia propones?

Cómo lo explicaríamos La resistencia recibe 1,3 V. R = 1,3 / 0,005 = 260 ohm. Con 330 ohm, la corriente ideal sería 1,3 / 330 = 3,94 mA. La potencia del resistor sería aproximadamente 5,1 mW. Son cifras del modelo de aula; se comprueban con las piezas reales.

$$R = \frac{V_{GPIO} - V_F}{I}, \qquad I_{330} = \frac{3.3 - 2.0}{330} \approx 3.94\,\text{mA}$$

Se elige un valor comercial razonable y se comprueba el conjunto, sin tratar el límite absoluto de un pin como corriente de trabajo.

Medimos tensión en paralelo con el componente. Para comenzar basta medir la caída en la resistencia y calcular su corriente. No se coloca el multímetro en modo corriente directamente entre alimentación y GND: esa conexión altera radicalmente el circuito. El docente muestra las puntas, la función y los puntos de medida antes de energizar. El registro debe indicar qué dos nodos se compararon, porque "medí 1,3 V" sin esa referencia no explica el circuito.

Ficha del primer LED

Dato	Predicción de aula	Dato del equipo
Resistencia	330 ohm	Leer y medir
Caída en resistor	1,3 V ideal	Medir nodos
Corriente	3,94 mA ideal	Calcular con V/R
Estado lógico	HIGH / LOW	Observar salida

El recorrido del programa hasta el pin

Clase 01 · Electrónica con ESP32: del circuito al primer programa · 4/6

VS Code es el editor; PlatformIO organiza el proyecto y sus herramientas; Arduino es el framework que aporta funciones de programación. El archivo platformio.ini declara placa, plataforma y bibliotecas. El curso incluye una versión de referencia fijada para ESP32 clásico. Compilar convierte el programa en firmware; cargarlo lo transfiere a la placa; abrir el monitor serie permite observar mensajes. Un error de compilación y un circuito mal conectado requieren comprobaciones diferentes.

Fragmento del primer ejemplo

```
constexpr uint8_t LED = 23;
uint32_t last = 0;
bool on = false;
void setup() {
  pinMode(LED, OUTPUT);
}
void loop() {
  uint32_t now = millis();
  if (uint32_t(now-last) >= 500) {
    last = now;
    on = !on;
    digitalWrite(LED, on);
  }
}
```

setup() prepara la ejecución y loop() vuelve a evaluar la lógica. El nombre LED ayuda a relacionar el esquema con un GPIO concreto. La función digitalWrite() ordena un nivel; el camino eléctrico determina qué ocurre con ese nivel. La versión completa c01_led también emite mensajes. En esta clase reconocemos el patrón temporal; en la tercera explicaremos por qué permite atender otros eventos. Antes de cambiarlo, predice cuánto dura cada estado y cuánto dura el ciclo completo.

Laboratorio: medir, ejecutar y explicar

Clase 01 · Electrónica con ESP32: del circuito al primer programa · 5/6

1. Identifica módulo, placa y GPIO23. Revisa la configuración de referencia antes de compilar c01_led.
2. Dibuja el camino de corriente. Coloca LED y resistencia con la alimentación desconectada.
3. Comprueba continuidad del protoboard y polaridad. Pide una revisión cruzada del esquema.
4. Ejecuta primero la versión equivalente en Wokwi y localiza qué variable representa cada componente.
5. En hardware, mide tensión en el resistor y calcula corriente. Anota qué valor de resistencia utilizaste.
6. Cambia el tiempo de 500 a 250 ms. Predice el periodo total antes de observarlo.
7. Conserva esquema, código y una explicación individual del resultado.

Un parpadeo correcto demuestra que esa configuración produjo el comportamiento observado. Para aprender más, cambia una sola condición: tiempo, valor de resistencia o pin documentado. La misma comprobación se hace con el código y con el circuito delante. Si el LED permanece apagado, comienza por alimentación, referencia, polaridad y camino de corriente. Si los mensajes cambian pero la luz no, ya existe una pista que permite separar la ejecución del programa de su salida física.

Aplicación a drones y lectura de investigación

Clase 01 · Electrónica con ESP32: del circuito al primer programa · 6/6

Un sistema UAV utiliza indicadores para distinguir alimentación, enlace, registro o estado del equipo. Un LED aislado puede indicar que hay energía, pero el software puede usarlo para comunicar otra condición. En una estación de banco, un patrón lento podría significar espera y otro representar una lectura pendiente. Esa convención debe estar documentada: el color no explica automáticamente el estado. El alumno propone un indicador para una función del mercado de MECRO y describe qué señal necesitaría para activarlo.

Del circuito sencillo al sistema

Hoy	Aplicación posterior	Qué añade el sistema
GPIO y LED	Indicador de estado	Condición documentada
GND y tensión	Interfaz de sensor	Niveles compatibles
Resistor calculado	Etapa de salida	Balance de potencia

Para investigar, localiza en la guía de tu placa la función del USB, de EN y de BOOT. Distingue alimentación del módulo y señal de programación. Después revisa la ficha de un LED concreto y busca la condición de corriente a la que se especifica su caída típica. El objetivo no es memorizar toda la hoja de datos, sino reconocer que una cifra técnica viene acompañada de una condición.

Comprobación final

¿HIGH equivale a una fuente ideal que alimenta cualquier carga?

Cómo lo explicaríamos HIGH expresa un estado de salida dentro de límites eléctricos. La corriente y la carga deben ser compatibles con el GPIO y con el circuito.

Entrega de la clase

Esquema del LED, cálculo con unidades, primer firmware y registro de observación.

Fuentes para ampliar

[MECRO] MECRO. Laboratorio Electrónica con ESP32: veinte proyectos.

[BOARD] Espressif Systems. ESP32-DevKitC V4: guía de la placa.

[GPIO] Espressif Systems. GPIO y restricciones de ESP32 clásico.

[PIO] PlatformIO. Plataforma Espressif 32, versión fijada 6.9.0.

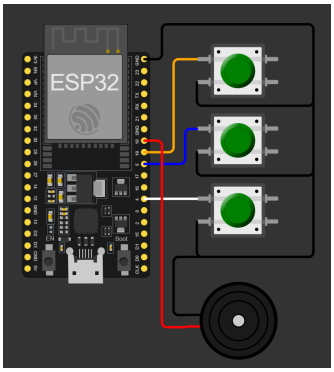
[MIT] Agarwal y Lang / MIT OpenCourseWare. 6.002 Circuits and Electronics; texto: Foundations of Analog and Digital Electronic Circuits (2005).

Entrada, decisión y salida

Clase 02 · Programación y entradas digitales: botones que significan eventos · 1/6

Al terminar Organizar un programa con funciones y detectar una pulsación sin confundir nivel, transición y rebote.

Un programa de hardware transforma observaciones en decisiones. El pulsador es la entrada; una condición representa la decisión; una luz o un sonido comunica la salida. Antes de escribir instrucciones, define el comportamiento: “mientras mantengo pulsado, la luz permanece encendida” es diferente de “cada pulsación cambia el estado”. La primera regla usa un nivel; la segunda necesita reconocer un evento. Si no se aclara esa diferencia, el programa puede parecer defectuoso aunque ejecute exactamente lo que se escribió.



MECRO, proyecto 02. Esquema original del laboratorio para reconocer la arquitectura en Wokwi. Para montaje físico se aplica la tabla y las correcciones de esta lectura.

El proyecto del piano permite observar esta cadena con tres entradas. Cada botón selecciona una acción. El dado digital añade una variable que conserva un resultado después de la pulsación. Reutilizaremos ambos como ejercicios de programación; no es necesario repetir dos montajes completos para enseñar el mismo concepto. Cada estudiante modifica una regla y debe explicar cuándo se evalúa.

Tipos, condiciones y funciones que ayudan a pensar

Clase 02 · Programación y entradas digitales: botones que significan eventos · 2/6

Una constante nombra una elección que no cambia durante la ejecución: por ejemplo, el número de pin. Una variable conserva información que sí puede cambiar. bool representa dos estados lógicos; un entero representa un conteo; float permite valores fraccionarios con precisión limitada. El tipo también afecta las operaciones. Si dividimos dos enteros, el resultado sigue reglas de aritmética entera. Escribir la unidad junto al nombre, como tiempo_ms o distancia_cm, reduce confusiones sin exigir un lenguaje más complejo.

Una intención y su representación

Intención	Ejemplo	Uso
Pin fijo	constexpr uint8_t BTN=18	Relacionar esquema y código
Estado	bool encendido=false	Conservar una decisión
Tiempo	uint32_t ahora=millis()	Medir intervalos
Magnitud fraccional	float distancia_cm	Expresar una lectura

Una función agrupa una tarea con entradas y una salida comprensibles. Una función llamada actualizarIndicador() explica mejor la intención que copiar varias instrucciones dispersas. El alcance de una variable define desde dónde se puede usar. Una variable creada dentro de loop() puede reiniciarse al repetir la función; si debe conservar el estado anterior, hay que elegir una duración de almacenamiento adecuada. El curso usa variables globales pequeñas y explícitas antes de introducir arquitecturas más avanzadas.

División que cambia el resultado

¿Cuánto vale 5/2 en una operación entre enteros y cómo obtienes 2,5?

Cómo lo explicaríamos Entre enteros da 2. Una expresión como 5.0f/2.0f conserva la fracción. La decisión depende de qué magnitud quieres representar.

Pull-up y rebote: dos problemas diferentes

Clase 02 · Programación y entradas digitales: botones que significan eventos · 3/6

Una entrada sin referencia puede responder a interferencias y mostrar estados variables. INPUT_PULLUP habilita una resistencia interna que tiende a llevar el pin a HIGH. El pulsador lo conecta a GND al presionarlo, así que pulsado se interpreta como LOW. Esa inversión es una convención del circuito. No se corrige cambiando cables al azar: se relacionan nivel físico y significado en una tabla de verdad.

Pulsador entre GPIO18 y GND

Situación	Lectura	Significado elegido
Abierto	HIGH	Reposo
Presionado	LOW	Solicitud
Lecturas alternantes breves	HIGH / LOW	Posible rebote

El contacto mecánico puede abrir y cerrar varias veces durante una sola acción. El pull-up fija el nivel cuando está abierto, pero no elimina ese rebote. Una estrategia sencilla acepta una lectura nueva solo cuando permanece estable durante un intervalo. El valor de 25 ms del ejemplo es una decisión de aula para comprobar, no una constante universal de todos los pulsadores. Al medir la respuesta importa también la latencia que esa espera introduce.

Una pulsación, varios cambios

Un contador aumenta cinco veces al tocar una vez el botón. ¿Basta cambiar INPUT por INPUT_PULLUP?

Cómo lo explicaríamos Primero se necesita una referencia válida. Si ya existe, hay que distinguir transición y rebote: contar una transición estable, no cada recorrido de loop() con el botón pulsado.

Lectura del antirrebote incluido

Clase 02 · Programación y entradas digitales: botones que significan eventos · 4/6

Núcleo de c02_boton

```
bool raw = digitalRead(BTN);
if (raw != rawPrevious) {
  rawPrevious = raw;
  changed = now;
}
if (uint32_t(now-changed) >= 25
    && raw != stable) {
  stable = raw;
  digitalWrite(LED, stable == LOW);
}
```

raw es la lectura instantánea. rawPrevious permite reconocer que cambió. changed conserva cuándo se produjo el cambio más reciente. stable representa lo que el programa acepta después de la espera. Durante esa espera, loop() puede seguir atendiendo otras tareas. Al separar estas variables evitamos llamar “estado del botón” a tres cosas que no significan lo mismo. La versión completa emite PRESS y RELEASE para observar ambas transiciones.

El piano original usa prioridades con if y else if: si se pulsan dos botones, la primera condición verdadera domina. Esa decisión puede ser correcta para un instrumento monofónico. Para una interfaz de mando hay que documentar qué ocurre con órdenes simultáneas. El alumno prueba dos entradas y describe el resultado antes de proponer otra prioridad. Programar también consiste en resolver esas situaciones límite.

Laboratorio y reto del dado

Clase 02 · Programación y entradas digitales: botones que significan eventos · 5/6

1. Conecta GPIO18 a GND mediante un pulsador; utiliza INPUT_PULLUP.
2. Ejecuta c02_boton y relaciona PRESS y RELEASE con la acción física.
3. Mantén pulsado dos segundos: verifica que no aparezcan nuevas pulsaciones aceptadas.
4. Añade un contador que aumente únicamente al aceptar stable == LOW.
5. En el piano del laboratorio, predice la prioridad de dos teclas y compruébala.
6. Como reto, usa el evento para solicitar un nuevo valor del dado y conserva el resultado.
7. Documenta las resistencias por segmento si se propone montar un display desnudo en hardware.

La simulación del dado sirve para comprender condiciones, variables y presentación. Un display real puede ser de ánodo o cátodo común y necesita limitar corriente por segmento según su montaje. La ilustración original del laboratorio no sustituye esa revisión. Por eso el reto obligatorio evalúa primero la lógica y la tabla de segmentos. La construcción física se realiza con un esquema eléctrico completo revisado por el docente.

Un contrato para las entradas humanas

Clase 02 · Programación y entradas digitales: botones que significan eventos · 6/6

En un banco relacionado con UAV, una pulsación puede solicitar una prueba, reconocer un aviso o cambiar la pantalla. Define el estado inicial, la acción permitida y qué sucede si el botón queda retenido. La interfaz debe poder explicarse a otra persona. Una buena prueba incluye pulsación breve, larga, doble, simultánea y al iniciar el equipo. Ninguna de esas pruebas requiere un dron en vuelo; se aprende con una entrada y una salida observables.

Casos de prueba de la interfaz

Prueba	**Resultado que se define**
Mantener pulsado	Un evento o acción sostenida
Dos botones	Prioridad o combinación
Pulsado al arrancar	Estado inicial documentado
Rebote	Una transición aceptada

Investiga la diferencia entre un botón con resistencia externa y un pin con resistencia interna habilitada. En el ESP32 clásico, GPIO34-39 son entradas y carecen de pull-up o pull-down internos. La elección del pin, por tanto, cambia el circuito necesario. Relaciona esa información con el hecho de que esos mismos pines pueden resultar útiles para lecturas analógicas. El nombre “GPIO” no promete exactamente las mismas funciones en todos los números.

Entrega de la clase

Código del botón, tabla nivel/evento y cuatro pruebas de interfaz.

Fuentes para ampliar

[MECRO] MECRO. Laboratorio Electrónica con ESP32: veinte proyectos.

[GPIO] Espressif Systems. GPIO y restricciones de ESP32 clásico.

[TIME] Arduino. Referencia del lenguaje: millis.

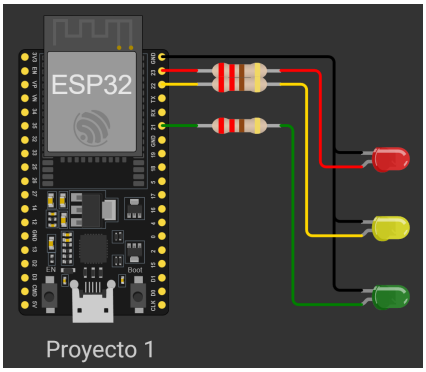
[MIT] Agarwal y Lang / MIT OpenCourseWare. 6.002 Circuits and Electronics; texto: Foundations of Analog and Digital Electronic Circuits (2005).

Por qué un programa correcto puede responder tarde

Clase 03 · Temporización y máquinas de estados: atender más de una tarea · 1/6

Al terminar Construir una secuencia que siga atendiendo eventos mientras transcurre un intervalo.

delay() expresa una espera cómoda cuando solo importa una secuencia. Mientras esa función bloquea el recorrido principal, no se ejecutan las otras instrucciones de loop(). Algunos periféricos e interrupciones pueden seguir funcionando, pero la lógica que dejaste después de la espera debe aguardar. Si un aviso requiere observar el botón durante una secuencia de tres segundos, una espera larga puede retrasar esa observación. El problema se descubre definiendo una tarea adicional, no prohibiendo una función por su nombre.



MECRO, proyecto 01. Esquema original del laboratorio para reconocer la arquitectura en Wokwi. Para montaje físico se aplica la tabla y las correcciones de esta lectura.

El semáforo del laboratorio ya usa millis() y switch. Lo conservamos y explicamos antes de aumentar su complejidad. Caja fuerte, detector de intrusos y ascensor ofrecen variantes de eventos y estados. En una misma clase no construiremos cuatro mecanismos: compararemos sus lógicas y cada equipo implementará una sola variación. Lo aprendido debe transferirse a un nuevo caso en lugar de convertirse en una lista de programas copiados.

Tiempo transcurrido y retorno del contador

Clase 03 · Temporización y máquinas de estados: atender más de una tarea · 2/6

millis() informa un contador de milisegundos desde el inicio. Para decidir si venció un intervalo, calculamos la resta entre el valor actual y el guardado. Con un tipo sin signo de 32 bits, esa resta permite tratar correctamente un retorno del contador para los intervalos cortos del curso. La comparación directa contra un instante futuro calculado con suma puede comportarse mal al atravesar ese retorno. No se trata de memorizar una frase: se puede comprobar con números pequeños y después con valores cercanos al límite.

$$\Delta t = \text{uint32}(t_{ahora} - t_{anterior})$$

Todos los tiempos de esta comparación usan el mismo tipo y la misma unidad.

Patrón temporal

```
uint32_t now = millis();
if (uint32_t(now-last) >= interval_ms) {
  last = now;
  actualizarTarea();
}
```

Guardar last = now evita intentar recuperar todas las ejecuciones perdidas después de una demora. Actualizar last += interval_ms responde a otra política de agenda. Primero elegiremos una tarea y una política explícitas. Para medir RPM más adelante, usaremos el tiempo realmente transcurrido; para un modelo matemático de paso fijo, el tiempo del modelo tendrá su propio significado. Dos relojes con la misma unidad pueden representar referencias distintas.

Estado, evento y transición

Clase 03 · Temporización y máquinas de estados: atender más de una tarea · 3/6

Un estado conserva qué situación del sistema está activa. Un evento señala algo que ocurrió o una condición que acaba de cumplirse. La transición define a qué estado se pasa y qué acciones se ejecutan. En el semáforo, ROJO, VERDE y AMARILLO son estados. “Han transcurrido tres segundos” es una condición de transición. Una solicitud peatonal puede quedar guardada para atenderse cuando corresponda. Separar estas ideas permite dibujar la lógica antes de escribir switch.

Fragmento de la secuencia

Estado	Condición	Siguiente estado
ROJO	3 s transcurridos	VERDE
VERDE	3 s transcurridos	AMARILLO
AMARILLO	1 s transcurrido	ROJO

Solicitud durante verde

Se pulsa el botón durante verde. ¿Dónde conservas la solicitud hasta llegar a rojo?

Cómo lo explicaríamos En una variable de solicitud persistente. El evento no se conserva si solo se pregunta por el nivel instantáneo al llegar a rojo. La variable se limpia después de atender la condición definida.

Una variable posicionActual en un ascensor de pasos puede guardar la posición ordenada sin haber medido el movimiento. Su nombre no confirma que una cabina haya llegado. En el curso usamos esa diferencia para distinguir estado de software y observación del objeto. La variante de ascensor queda como estudio de lógica; un montaje real necesitaría resolver referencia inicial, driver, sensores y límites del mecanismo.

Lectura de una transición pequeña

Clase 03 · Temporización y máquinas de estados: atender más de una tarea · 4/6

Núcleo de c03_estados

```
if (digitalRead(BTN) == LOW) request = true;
if (state == State::GREEN
    && uint32_t(now-since) >= 3000) {
  enter(State::YELLOW, now);
}
```

La función enter() centraliza la asignación del estado, la referencia temporal y las salidas. Así no quedan luces con valores heredados de otra rama. El programa de referencia registra la solicitud y no obliga a que el usuario siga pulsando hasta el momento de atención. En una interfaz real se puede añadir el antirrebote de la clase anterior. Ambas funciones tienen propósitos distintos: una define la secuencia y la otra interpreta el contacto.

Para depurar, construye una tabla con tiempo, estado, entrada y acción. No es necesario imprimir miles de mensajes por segundo. Registra transiciones y hechos importantes para que el monitor permita reconstruir la secuencia. Si cada recorrido de loop() emite una línea, la propia comunicación puede ocupar una parte importante del tiempo y esconder lo que intentabas observar. Un registro útil se diseña alrededor de preguntas concretas.

Laboratorio de respuesta y variación

Clase 03 · Temporización y máquinas de estados: atender más de una tarea · 5/6

1. Dibuja la tabla de transiciones y señala qué variables deben conservarse.
2. Ejecuta el semáforo y pulsa en cada uno de sus tres estados.
3. Añade un contador de solicitudes atendidas sin detener la secuencia.
4. Compara una versión con delay(3000) y otra por tiempo transcurrido.
5. Introduce una acción de parpadeo independiente para mostrar dos agendas.
6. Elige caja fuerte o PIR como variante y describe su estado inicial y su transición de retorno.
7. Conserva un registro de eventos que permita explicar una prueba completa.

Registro de prueba

Tiempo / ms	Estado	Evento / acción
Inicio	Por registrar	Estado inicial
Solicitud	Por registrar	Entrada observada
Transición	Por registrar	Cambio de salidas
Atención	Por registrar	Solicitud consumida

La prueba busca una diferencia de comportamiento: por ejemplo, qué sucede con una solicitud breve que aparece durante una espera. Observar solo que las luces completan el ciclo no responde esa pregunta. Si el equipo afirma que la respuesta es “inmediata”, debe decir cómo la observó y qué demora podría no detectar con ese método. En esta etapa basta demostrar que el evento se conserva y se atiende según una regla.

Tres estados para un banco de sensores

Clase 03 · Temporización y máquinas de estados: atender más de una tarea · 6/6

Una aplicación cercana al mercado UAV puede usar INICIO, LISTO y FALLO. En INICIO se preparan los periféricos; en LISTO se adquieren y muestran datos; en FALLO se informa una condición que impide tratarlos como recientes. El estado describe el sistema, mientras un LED u OLED lo presenta. La misma estructura permitirá que la clase final detecte un sensor ausente sin seguir mostrando una lectura antigua como si fuera nueva.

Transferencia de la lógica

Proyecto	Evento	Qué se conserva
Caja fuerte	Clave aceptada	Estado de acceso
PIR	Cambio de movimiento	Estado de aviso
Banco UAV	Lectura vencida	Estado de datos

Investiga en una biblioteca si sus funciones esperan a terminar una operación. Una interfaz aparentemente sencilla puede ocultar una espera. Identifica el comportamiento de motor.step() en la variante del ascensor y de pulseIn() en el sensor ultrasónico. Una arquitectura sin delay() todavía puede bloquear en otras llamadas. La habilidad es reconocer la duración y el lugar de la espera, no buscar una palabra prohibida.

Entrega de la clase

Tabla de estados, variante temporal y registro de atención de eventos.

Fuentes para ampliar

[MECRO] MECRO. Laboratorio Electrónica con ESP32: veinte proyectos.

[TIME] Arduino. Referencia del lenguaje: millis.

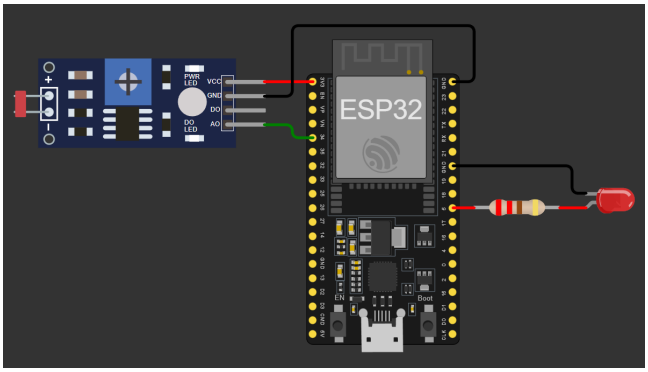
[UART] Espressif Systems. Arduino-ESP32: Serial/UART.

Qué convierte un ADC

Clase 04 · ADC y calibración: de una lectura a una decisión · 1/6

Al terminar Interpretar lecturas analógicas, construir una escala de trabajo y justificar dos umbrales de histéresis.

Un sensor analógico representa una condición mediante una tensión. El convertidor analógico a digital devuelve un número que depende de esa tensión y de la configuración. En una lectura de 12 bits existen 4096 códigos posibles, numerados de 0 a 4095. Esa cantidad describe resolución de código; no demuestra exactitud ni una relación perfectamente lineal en toda la entrada. El ESP32 clásico ofrece varios canales y restricciones que hay que relacionar con el pin elegido.



MECRO, proyecto 03. Esquema original del laboratorio para reconocer la arquitectura en Wokwi. Para montaje físico se aplica la tabla y las correcciones de esta lectura.

GPIO34 pertenece a ADC1 y es una entrada sin pull-up interno. En nuestro banco se conecta a un potenciómetro o a la salida analógica de un divisor de luz que respete el rango del pin. Elegimos ADC1 para que la práctica se pueda conservar cuando añadamos Wi-Fi. La elección no convierte el pin en tolerante a 5 V. La configuración de atenuación y el rango útil se consultan para la variante real.

La LDR cambia su resistencia con la iluminación. El divisor la transforma en tensión. Si se intercambia la posición de la LDR y del resistor fijo, se invierte la tendencia de la lectura. Por eso no se enseña que oscuridad equivale siempre a un número pequeño. Se sigue el circuito y se mide la tendencia. Ese razonamiento también sirve al interpretar un joystick o cualquier sensor con salida analógica.

La escala ideal y la escala calibrada

Clase 04 · ADC y calibración: de una lectura a una decisión · 2/6

$$\widehat{V} = \frac{N}{4095} V_{FS}$$

Esta relación describe un ADC ideal con escala VFS definida. No declara el rango real ni la exactitud del ESP32.

En un modelo ideal con escala completa de 3,0 V, el código 2048 representa aproximadamente 1,50 V. En hardware comparamos esa idea con una medición y con la API calibrada disponible. analogRead() devuelve código bruto y analogReadMilliVolts() ofrece una conversión calibrada según el soporte de la plataforma. Una variable llamada voltaje no queda calibrada por recibir ese nombre. El alumno conserva el método de obtención junto al dato.

Cuatro conceptos distintos

Concepto	Qué describe	Ejemplo de comprobación
Resolución	Cantidad de códigos	12 bits: 4096 códigos
Exactitud	Cercanía a referencia	Comparar con instrumento
Repetibilidad	Dispersión al repetir	Varias lecturas iguales
Saturación	Extremo de conversión	Más entrada, mismo código

Para construir una escala de trabajo, observa dos condiciones reproducibles. En la LDR pueden ser dos niveles de iluminación del banco; en un joystick, centro y recorridos permitidos. Los valores se guardan como datos de ese montaje. No se presentan como propiedades universales del componente. Conviene evitar las zonas donde la lectura se satura y perdería capacidad para distinguir cambios.

Ruido, promedio e histéresis

Clase 04 · ADC y calibración: de una lectura a una decisión · 3/6

Una lectura real puede fluctuar aunque el objeto parezca quieto. Un promedio reduce algunas variaciones rápidas, pero también retrasa cambios y no corrige un sesgo de calibración. La histéresis usa dos límites para evitar que una salida cambie repetidamente cuando la entrada oscila cerca de una frontera. Si el sistema ya está encendido, requiere superar el umbral de apagado para cambiar de estado; entre ambos conserva su decisión.

Lecturas cerca del umbral

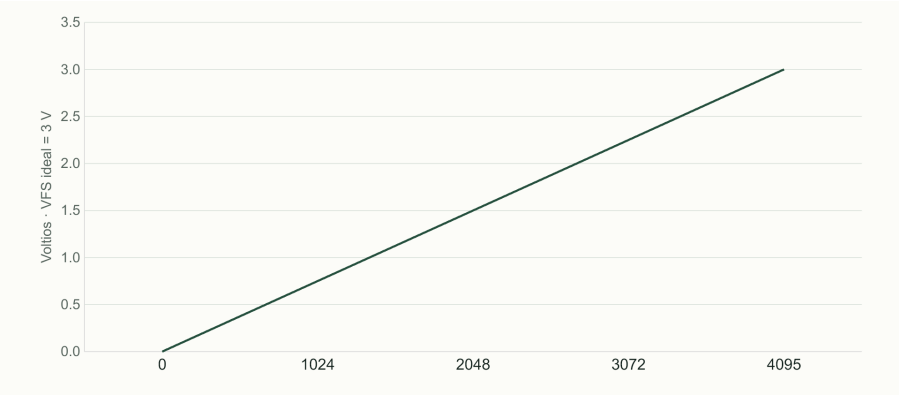
La secuencia es 1490, 1510, 1495, 1505, 1820. El sistema inicia apagado, enciende por debajo de 1500 y apaga por encima de 1800. ¿Qué estados aparecen?

Cómo lo explicaríamos Enciende con 1490; mantiene encendido con 1510, 1495 y 1505; apaga con 1820. El intervalo intermedio conserva memoria del estado.

Operaciones sobre la lectura

Operación	Ayuda a	No resuelve por sí sola
Promedio	Atenuar fluctuación	Calibración absoluta
Histéresis	Evitar conmutación repetida	Cambiar polaridad del divisor
Límite de rango	Detectar o recortar extremos	Recuperar información saturada

Antes de filtrar, busca si la variación corresponde al objeto o al montaje. Mover un cable defectuoso no produce el mismo problema que una señal física cambiante. En el registro se conservan lectura bruta y resultado procesado. Si solo guardas el promedio, será difícil investigar qué sucedió con la entrada original.



Modelo ideal para aprender la escala. No representa calibración ni exactitud del ADC físico.

Código que conserva una decisión

Clase 04 · ADC y calibración: de una lectura a una decisión · 4/6

Núcleo de c04_adc

```
int raw = analogRead(34);
if (raw < 1500) on = true;
else if (raw > 1800) on = false;
digitalWrite(23, on);
```

Entre ambos umbrales no se asigna un valor nuevo a on. Esa ausencia es deliberada: la variable mantiene el estado anterior. Cambiar else if por otra regla independiente puede modificar el comportamiento si las condiciones se solapan. La versión completa configura resolución y atenuación y registra tiempo, código y estado. El periodo de 50 ms se declara para que los datos puedan compararse.

map() realiza una transformación de escala con aritmética entera y no limita automáticamente la salida. Si un valor excede los extremos de entrada, el resultado puede exceder los extremos deseados. Cuando el proyecto necesita recortar, se hace explícitamente y se conserva el hecho de que la entrada salió del rango. Una escala de porcentaje es una convención de trabajo; no equivale automáticamente a una medida física certificada de iluminación o humedad.

Laboratorio con una referencia visible

Clase 04 · ADC y calibración: de una lectura a una decisión · 5/6

1. Verifica la alimentación y la referencia del divisor o potenciómetro.
2. Registra diez lecturas con la entrada quieta y anota mínimo, máximo y contexto.
3. Repite en dos condiciones distintas y determina el sentido de la respuesta.
4. Ejecuta c04_adc y sustituye los umbrales de ejercicio por una propuesta de tu montaje.
5. Introduce cambios lentos alrededor de la frontera y compara uno y dos umbrales.
6. Conserva los datos brutos junto a la salida; comprueba qué sucede al desconectar una entrada de forma controlada.
7. Explica una limitación del modelo de Wokwi frente al circuito físico.

La simulación permite controlar una entrada y estudiar la lógica del programa. El comportamiento analógico del mundo real añade dispersión, impedancias, alimentación y límites que el modelo no reproduce completamente. Una práctica de calibración física requiere una referencia y condiciones conservadas. El alumnado sin kit puede resolver la parte lógica y analizar un conjunto de datos identificado, mientras el docente demuestra la medición real.

Aplicación: observar batería sin conectar una batería al ADC

Clase 04 · ADC y calibración: de una lectura a una decisión · 6/6

El principio de un divisor permite reducir una tensión para observarla, pero una batería real de dron exige un diseño de entrada completo y conocer sus límites. En este bloque practicamos con una fuente de baja tensión o un potenciómetro. El equipo puede dibujar una cadena conceptual de observación de batería: magnitud, acondicionamiento, ADC, calibración y umbral. El aprendizaje transferible consiste en justificar cada transformación y detectar qué falta por comprobar.

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Los extremos pertenecen a una calibración declarada; si coinciden, la transformación es inválida.

Investiga qué ocurre con ADC2 del ESP32 clásico al usar Wi-Fi y por qué elegimos GPIO34. Localiza además qué expresa la atenuación. Presenta una lectura de la API en milivoltios con el método que la produjo, sin confundir el valor de alimentación de la placa con la escala exacta del convertidor. La conclusión de clase debe diferenciar una medida, una conversión ideal y una decisión de software.

Entrega de la clase

CSV de lecturas, sentido del divisor y justificación de umbrales.

Fuentes para ampliar

[MECRO] MECRO. Laboratorio Electrónica con ESP32: veinte proyectos.

[ADC] Espressif Systems. Arduino-ESP32: ADC.

[GPIO] Espressif Systems. GPIO y restricciones de ESP32 clásico.

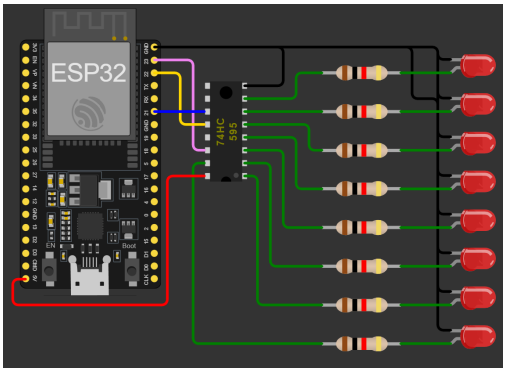
[MIT] Agarwal y Lang / MIT OpenCourseWare. 6.002 Circuits and Electronics; texto: Foundations of Analog and Digital Electronic Circuits (2005).

Una salida digital puede expresar más que encendido y apagado

Clase 05 · PWM, sonido y salidas digitales en serie · 1/6

Al terminar Distinguir frecuencia, resolución y ciclo de trabajo, y ampliar salidas con un registro de desplazamiento.

PWM alterna una salida entre sus niveles digitales siguiendo un periodo y una fracción de tiempo activa. La señal conserva esos niveles; lo que cambia es su distribución temporal. Un LED y su circuito pueden producir una apariencia de brillo distinta al variar esa fracción. Un instrumento o una carga pueden responder de otra manera. No convertimos automáticamente el GPIO en una fuente analógica de potencia variable.



MECRO, proyecto 04. Esquema original del laboratorio para reconocer la arquitectura en Wokwi. Para montaje físico se aplica la tabla y las correcciones de esta lectura.

El piano muestra una señal cuyo ritmo determina la nota, mientras el dimmer muestra una señal cuyo ciclo de trabajo regula el efecto de brillo. Son usos diferentes de una onda digital. El registro 74HC595 del proyecto de luces ofrece otra ampliación: convierte una secuencia de bits en varias salidas conservadas. Esta clase compara las dos ideas sin confundir “muchas salidas” con “resolución de PWM”.

Periodo, frecuencia y resolución

Clase 05 · PWM, sonido y salidas digitales en serie · 2/6

$$f = \frac{1}{T}, \qquad D = \frac{t_{on}}{T}, \qquad d \approx D(2^b - 1)$$

D es una proporción; d es un código de mando de b bits para la convención del ejemplo.

Una señal de 1 kHz

Con periodo de 1 ms y tiempo activo de 0,25 ms, ¿qué ciclo de trabajo y qué código de 8 bits propones?

Cómo lo explicaríamos D = 0,25, equivalente al 25 %. El código aproximado es 0,25 × 255 = 63,75: se elige 64. La frecuencia sigue siendo 1 kHz.

Tres controles diferentes

Cambio	Qué modifica	Ejemplo
Frecuencia	Repetición por segundo	Nota en buzzer pasivo
Duty	Fracción activa	Brillo del LED
Resolución	Pasos de mando	8 bits: 0 a 255

Más resolución permite seleccionar más códigos, pero el periférico debe generar la combinación de frecuencia y resolución solicitada. En el código de referencia se comprueba si la configuración tuvo éxito. Elegir una frecuencia nueva puede compartir recursos con otra salida, según la configuración del periférico y las bibliotecas. En la clase de servo volveremos a este punto con una señal de 50 Hz que tiene un significado distinto al brillo.

El 74HC595 y una palabra de ocho bits

Clase 05 · PWM, sonido y salidas digitales en serie · 3/6

El registro recibe bits en serie y mantiene un conjunto de salidas. DATA transporta el valor de cada bit; CLOCK determina cuándo se incorpora; LATCH presenta el conjunto almacenado. Un patrón como 00010000 puede representar una luz encendida entre ocho. Desplazar ese patrón produce la animación del proyecto. La operación shiftOut() genera una secuencia por software; no demuestra que el circuito esté usando automáticamente el periférico SPI del ESP32.

Señales del registro

Señal	Papel	Qué observa el alumno
DATA	Bit que se transmite	0 o 1
CLOCK	Paso de desplazamiento	Ocho pulsos por palabra
LATCH	Actualización de salidas	Cambio del patrón completo

La salida del integrado también tiene límites de corriente. En un montaje con LEDs se requiere una resistencia por rama. La alimentación del SN74HC595 y los umbrales de entrada se consultan juntos: un HC alimentado a 5 V no debe suponerse compatible en todas sus condiciones con una señal de 3,3 V. El ejemplo físico del curso se prepara a 3,3 V con corrientes de indicador pequeñas y una hoja de conexiones revisada.

Ocho indicadores

Con LED de 2,0 V y alimentación de 3,3 V, un resistor de 680 ohm produce aproximadamente 1,9 mA por rama en el modelo ideal. ¿Qué suma representan ocho ramas?

Cómo lo explicaríamos Aproximadamente 15,3 mA. También se revisan las condiciones por salida y del integrado completo; la suma de ramas no autoriza ignorar la caída de tensión del driver.

Un código y dos familias de API

Clase 05 · PWM, sonido y salidas digitales en serie · 4/6

Uso de la capa de compatibilidad incluida

```
CoursePwm led(23, 0);
bool ok = led.begin(1000, 8);
if (ok) {
  uint32_t duty = (raw * 255UL) / 4095UL;
  led.write(duty);
}
```

La capa course_pwm.h conserva separados el número de GPIO y el canal. En Arduino-ESP32 2.x se configuran canal y asociación de pin; en 3.x cambió la API y varias operaciones se expresan por pin. El material fija un entorno de referencia para los ejemplos y explica esa diferencia. Encontrar una función con el mismo nombre no garantiza que sus argumentos conserven el significado de una versión anterior.

Patrón de una actualización del registro

```
digitalWrite(LATCH, LOW);
shiftOut(DATA, CLOCK, MSBFIRST, pattern);
digitalWrite(LATCH, HIGH);
```

El orden de bits se relaciona con la posición física de las salidas. Si la animación parece invertida, primero compara el patrón, la convención MSBFIRST y las conexiones. No hace falta cambiar simultáneamente ocho cables y todo el programa para investigar una inversión. Una prueba de un solo bit permite identificar la correspondencia.

Laboratorio: un efecto y una secuencia

Clase 05 · PWM, sonido y salidas digitales en serie · 5/6

1. Ejecuta c05_pwm con LED y resistencia; reconoce pin, canal, frecuencia y resolución.
2. Selecciona códigos 0, 64, 128, 192 y 255. Predice sus fracciones de mando.
3. Si hay analizador lógico u osciloscopio, mide periodo y tiempo activo; conserva la configuración.
4. En el proyecto 4, prueba un bit cada vez y dibuja su relación con las salidas.
5. Compara cambiar frecuencia del buzzer con cambiar duty del LED.
6. Conserva una tabla que diferencie mando calculado, señal observada y efecto visible.

Mando del ejercicio

Código / 8 bits	Fracción aproximada	Observación
64	25,1 %	Por comprobar
128	50,2 %	Por comprobar
192	75,3 %	Por comprobar

La vista de una simulación ayuda a relacionar el algoritmo con el patrón. Una estimación visual de brillo no es una medida lineal de potencia óptica. Si el equipo usa solo observación, puede describir orden y contraste entre estados. Para afirmar valores de señal necesita el instrumento o el modelo identificado. El docente demuestra al menos una lectura de periodo para ligar la fórmula con un objeto real.

Cómo se transfiere la idea a un UAV

Clase 05 · PWM, sonido y salidas digitales en serie · 6/6

En una aeronave conviven interfaces digitales con significados distintos. Una orden hacia un servo, un controlador de motor y una cadena de LEDs no se interpreta del mismo modo. Compartir el término PWM no hace intercambiables sus frecuencias, anchos de pulso ni etapas de potencia. El aprendizaje del curso es leer qué espera el dispositivo receptor y comprobar la señal generada antes de integrarlo. No se usa el pin como sustituto de un ESC.

Qué veremos después

Dispositivo	Dato relevante	Comprobación
LED	Duty del efecto	Periodo y estado
Servo	Ancho de pulso	Intervalo permitido
Driver DC	Entrada de mando	Potencia y respuesta

Investiga en la hoja del SN74HC595 la diferencia entre registro de desplazamiento y registro de salida. Busca además sus condiciones de alimentación y de corriente. Compara la comunicación DATA/CLOCK/LATCH con un bus SPI de propósito general: ambos son síncronos, pero la función del dispositivo y el protocolo concreto siguen importando. Presenta una semejanza y una diferencia útiles para cablear.

Entrega de la clase

Tabla de señales PWM y correspondencia de bits del proyecto 4.

Fuentes para ampliar

[MECRO] MECRO. Laboratorio Electrónica con ESP32: veinte proyectos.

[PWM] Espressif Systems. Arduino-ESP32: LEDC.

[MIG] Espressif Systems. Migración Arduino-ESP32 2.x a 3.0.

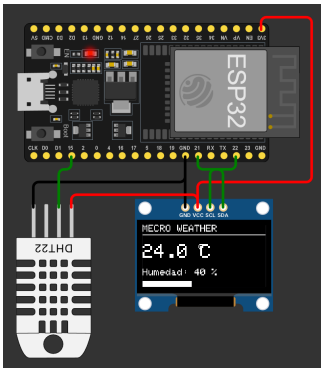
[HC595] Texas Instruments. SN74HC595: hoja de datos.

El valor no viaja solo

Clase 06 · Sensores digitales, I2C y pantallas: un dato con identidad · 1/6

Al terminar Leer y presentar una variable conservando unidad, instante, estado de la lectura y dirección del dispositivo.

Una lectura de temperatura necesita unidad, instante y origen. El número 24 puede significar grados Celsius, un código de conversión o un valor anterior que quedó en la pantalla. El proyecto de estación de clima permite comenzar con esa distinción. El sensor adquiere; el programa interpreta; la pantalla presenta. Una actualización de pantalla no demuestra que haya ocurrido una adquisición nueva.



MECRO, proyecto 07. Esquema original del laboratorio para reconocer la arquitectura en Wokwi. Para montaje físico se aplica la tabla y las correcciones de esta lectura.

Separar tareas ayuda a organizarlas a velocidades distintas. Un sensor lento puede actualizarse cada cierto intervalo mientras la interfaz responde a un botón más a menudo. La última lectura se almacena junto a un indicador de validez y su tiempo. Cuando falla la adquisición, la aplicación puede mostrar FALLO o una lectura anterior con su antigüedad. Reemplazar todo error por cero confunde una ausencia de dato con una medida legítima.

La estación de clima del laboratorio usa un sensor DHT y una pantalla OLED. El sensor DHT no se convierte en I2C por compartir una placa con la pantalla. Cada conexión sigue su propio protocolo. Más adelante, la IMU sí compartirá I2C con una pantalla si direcciones y condiciones eléctricas son compatibles.

Qué ofrece un bus I2C

Clase 06 · Sensores digitales, I2C y pantallas: un dato con identidad · 2/6

I2C utiliza una línea de datos y una de reloj con dispositivos identificados mediante dirección. En el perfil clásico de las prácticas usamos SDA21 y SCL22, cuando ese montaje lo permita. Esas conexiones sustituyen el semáforo que usaba algunos de los mismos pines: los ejercicios son configuraciones diferentes, no un único circuito acumulado de manera automática. La dirección se comprueba con el dispositivo real y con su biblioteca.

Tres comunicaciones del banco

Interfaz	Organización	Ejemplo
UART	Flujo de bytes	Monitor serie
I2C	Bus con direcciones	OLED y MPU6050
DHT	Señal y temporización propias	Temperatura y humedad

Las líneas I2C necesitan condiciones de polarización compatibles. Algunos módulos ya incluyen resistencias pull-up y al conectarlos en paralelo cambia la resistencia equivalente. Lo importante para el primer montaje es identificar dónde van esas resistencias y a qué tensión se conectan. No se supone que cualquier módulo alimentado con 5 V produzca señales seguras para el ESP32. La ficha del módulo y las medidas del banco completan el esquema.

Dos dispositivos, una dirección

Dos módulos responden con la misma dirección fija. ¿Basta conectar ambos en paralelo y darles nombres distintos en C++?

Cómo lo explicaríamos No. Los nombres del programa no cambian la dirección del bus. Se revisan opciones de dirección o una arquitectura de bus adecuada.

Validez y antigüedad

Clase 06 · Sensores digitales, I2C y pantallas: un dato con identidad · 3/6

Un registro mínimo puede contener valor, unidad, tiempo e indicador válido. Para tratar antigüedad se resta el tiempo de la última adquisición aceptada al tiempo actual. El umbral permitido depende de la misión y de la velocidad del sensor. Un dato ambiental de hace un segundo y uno de orientación de hace un segundo pueden tener utilidades muy diferentes. El curso pide nombrar el criterio de cada uno.

$$edad_{ms} = t_{actual} - t_{ultima\ lectura\ valida}$$

El instante se actualiza al aceptar una lectura nueva, no simplemente al redibujar la pantalla.

Tres situaciones de presentación

Entrada	Pantalla útil	Registro
Lectura reciente	Valor y unidad	Instante nuevo
Lectura inválida	FALLO o aviso	Estado de error
Lectura antigua	Valor con edad	Última adquisición válida

En bibliotecas que representan una lectura inválida mediante NaN, una comparación numérica corriente puede no comportarse como se espera. Hay que consultar la condición de validez y tratarla antes de decidir una salida. El ejemplo de clima ya contiene una comprobación isnan(); la clase explica para qué sirve y qué debe conservar el resto de la aplicación cuando se activa.

Cómo investigar una conexión

Clase 06 · Sensores digitales, I2C y pantallas: un dato con identidad · 4/6

Fragmento de diagnóstico I2C

```
Wire.begin(21, 22);
Wire.beginTransmission(0x68);
uint8_t status = Wire.endTransmission();
bool acknowledged = (status == 0);
```

Una respuesta en la dirección 0x68 indica que algún dispositivo reconoció esa dirección en esa prueba. No certifica por sí sola la identidad, la escala ni la calidad de sus medidas. La identificación de registro o el procedimiento de la biblioteca aporta otra comprobación. El diagnóstico se realiza primero sin mezclarlo con pantallas complejas, gráficas y múltiples sensores.

Una pantalla OLED se organiza mediante un controlador y una biblioteca que traduce texto o píxeles a comandos. La dirección habitual del ejemplo puede diferir en otro módulo. También hay pantallas de aspecto parecido con controladores distintos. Antes de cambiar el código del sensor, el equipo identifica si el problema pertenece al bus, al controlador de pantalla o a la lógica de presentación.

Pantalla congelada

La pantalla sigue mostrando 24,0 °C después de desconectar el sensor. ¿Qué dato del software falta consultar?

Cómo lo explicaríamos La validez y el instante de la última adquisición. El valor mostrado puede ser una copia antigua; hace falta una política explícita para ese estado.

Laboratorio de adquisición y presentación

Clase 06 · Sensores digitales, I2C y pantallas: un dato con identidad · 5/6

1. Abre el proyecto 7 y localiza sensor, pantalla y protocolos.
2. Registra el tipo exacto de sensor que usa el archivo del simulador; no lo supongas por el título.
3. Localiza periodo de lectura, periodo de pantalla y condición de fallo.
4. Añade al registro una marca temporal y un estado de validez.
5. Identifica los dispositivos I2C y sus direcciones con la prueba preparada por el docente.
6. Provoca una ausencia de lectura en la simulación o con el procedimiento de banco indicado.
7. Explica cómo cambia la presentación y qué valor conserva el registro.

En un montaje físico se revisa el circuito desconectado y se aplica el procedimiento de comprobación del banco. Si el sensor no puede desconectarse durante operación, se simula la condición por software y se declara ese origen. La finalidad del ejercicio es que la aplicación distinga dato y estado. No se presenta una inyección de error como una medición de fallo real del componente.

De la estación ambiental a una carga de sensores

Clase 06 · Sensores digitales, I2C y pantallas: un dato con identidad · 6/6

Una carga de sensores para un UAV puede utilizar una arquitectura semejante: adquisición, procesamiento, almacenamiento y presentación. Los requisitos de tiempo y precisión cambian con la variable. La organización de módulos que aprendemos hoy permite añadir la IMU sin convertir todo el programa en una única función extensa. Cada dato llega acompañado de la información que permite usarlo con criterio.

Preguntas para cualquier sensor nuevo

Pregunta	Dónde buscar
¿Qué variable entrega?	Hoja de datos y biblioteca
¿Con qué unidad?	API y escala configurada
¿Cada cuánto es nueva?	Adquisición y marcas de tiempo
¿Cómo informa fallo?	Resultado de la operación

Investiga la diferencia entre dirección del bus, número de pin y nombre de la variable. Son referencias de niveles distintos. Para ampliar, compara UART, I2C y SPI con una tabla de líneas, organización y dispositivo de ejemplo. No necesitas implementar tres protocolos completos en una sesión: sí debes poder explicar por qué conectar la pantalla a un pin de sensor DHT no cumple la misma función.

Entrega de la clase

Mapa de interfaces y registro de valor, unidad, tiempo y estado.

Fuentes para ampliar

[MECRO] MECRO. Laboratorio Electrónica con ESP32: veinte proyectos.

[WIRE] Espressif Systems. Arduino-ESP32: I2C.

[UART] Espressif Systems. Arduino-ESP32: Serial/UART.

[IMU] Adafruit. MPU6050: biblioteca, cableado y lectura.

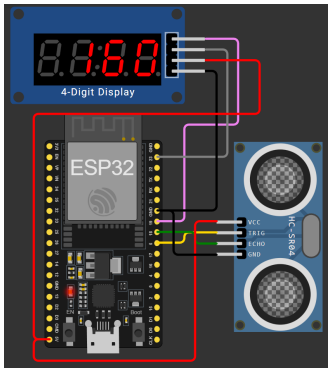
[WOKWI] Wokwi. ESP32 Simulation: periféricos y límites.

Qué observa el sensor ultrasónico

Clase 07 · Distancia: tiempo de vuelo, adaptación de nivel y lecturas inválidas · 1/6

Al terminar Convertir un tiempo de eco en distancia y comprobar su coherencia sin confundir falta de eco con distancia cero.

El HC-SR04 representa distancia mediante la duración de un pulso de eco asociado a una medición acústica. El sonido recorre la ida hasta el objeto y la vuelta al sensor. Esa doble trayectoria explica la división entre dos. No se está midiendo luz ni una distancia por imagen. El tamaño, orientación y superficie del objeto pueden cambiar el eco disponible en hardware.



MECRO, proyecto 06. Esquema original del laboratorio para reconocer la arquitectura en Wokwi. Para montaje físico se aplica la tabla y las correcciones de esta lectura.

En Wokwi se puede seleccionar una distancia de entrada del modelo y observar qué informa el programa. Eso es útil para comprobar la conversión y el tratamiento de extremos. En el banco, la distancia se contrasta con una referencia física y un objeto definido. La simulación no sustituye las comprobaciones de alimentación, nivel de señal, montaje o respuesta ante superficies diferentes.

Una fórmula con ida y vuelta

Clase 07 · Distancia: tiempo de vuelo, adaptación de nivel y lecturas inválidas · 2/6

$$d = \frac{c\,t}{2}, \qquad d_{cm} = \frac{0.0343\,t_{\mu s}}{2}$$

Se usa c = 343 m/s como aproximación de aula. Las condiciones del aire afectan a esa aproximación.

Eco de 1000 microsegundos

Con la velocidad de aula, ¿qué distancia representa un pulso de 1000 μs?

Cómo lo explicaríamos El recorrido total sería 34,3 cm. La distancia de ida es la mitad: 17,15 cm. Es una conversión del modelo, no una verificación de exactitud del módulo.

Tiempo, cálculo y referencia

Dato	Unidad	Qué debe acompañarlo
Duración del eco	microsegundos	Operación válida
Distancia calculada	cm	Conversión utilizada
Distancia de referencia	cm	Puntos físicos de medida

Una comparación requiere definir desde dónde se mide. La cara del sensor y el borde de una caja no son la misma referencia. Para una primera práctica bastan tres posiciones separadas y una superficie que el docente haya comprobado. Repetir lecturas en la misma posición permite observar dispersión. Cambiar posición y superficie a la vez impediría atribuir con claridad una diferencia.

La señal de 5 V y la entrada de 3,3 V

Clase 07 · Distancia: tiempo de vuelo, adaptación de nivel y lecturas inválidas · 3/6

El esquema original del laboratorio representa la simulación. Para hardware, el nivel de ECHO debe adaptarse a la entrada del ESP32 cuando el módulo lo entrega a 5 V. No se supone que un GPIO de la placa tolere ese nivel. En un divisor resistivo, R1 se coloca entre la salida ECHO y el nodo de medida; R2 se coloca entre ese nodo y GND. La referencia común debe estar definida.

$$V_{nodo} = V_{echo}\frac{R_2}{R_1 + R_2}$$

Ejemplo de banco: R1 = 2,2 kohm y R2 = 3,3 kohm producen 3,0 V nominales desde 5,0 V. Se verifica el nivel con el montaje real.

Interfaz física que se comprueba

Conexión	Función	Revisión
VCC del sensor	Alimentación del módulo	Ficha del módulo
TRIG	Inicio de medición	Compatibilidad de entrada
ECHO adaptado	Señal al GPIO	Nivel medido
GND	Referencia común	Continuidad

El divisor es parte de la interfaz, no una instrucción opcional que se omite porque la animación funciona. Un sensor de distancia compatible con lógica de 3,3 V puede utilizar otra interfaz, siempre que su documentación lo respalde. El docente conserva la referencia del componente físico: “HC-SR04 o equivalente” no hace idénticos todos sus módulos.

Tiempo de espera y ausencia de dato

Clase 07 · Distancia: tiempo de vuelo, adaptación de nivel y lecturas inválidas · 4/6

Núcleo de c07_distancia

```
unsigned long us = pulseIn(ECHO, HIGH, 25000UL);
if (us == 0) {
  Serial.println("INVALID");
  return;
}
float distance_cm = us * 0.0343f / 2.0f;
```

La llamada tiene un tiempo de espera explícito. Sigue siendo bloqueante, pero su duración máxima queda acotada para esta práctica. Devolver cero por no completar el pulso no significa que el objeto esté tocando el sensor. Primero se interpreta la condición de la operación y después se calcula distancia. Una pantalla que muestra 0 cm para cualquier error pierde esa distinción.

El tiempo de la adquisición también ocupa parte de la agenda. Si después coordinamos un servo, habrá que considerar su movimiento, la espera del sensor y el momento de aceptar la lectura. La versión de clase es deliberadamente pequeña para que esa duración pueda verse antes de construir un explorador. Guardar tiempo de inicio y de fin permite comprobar cuánto estuvo ocupada cada operación.

Laboratorio de contraste y fallos

Clase 07 · Distancia: tiempo de vuelo, adaptación de nivel y lecturas inválidas · 5/6

1. Revisa el esquema de simulación y la tabla de interfaz física.
2. Ejecuta c07_distancia y cambia tres distancias conocidas en Wokwi.
3. Calcula manualmente un caso a partir del tiempo de eco.
4. En banco, contrasta tres posiciones con regla y objeto definidos.
5. Retira el eco con el procedimiento del docente y observa INVALID.
6. Compara la conversión ideal y la referencia, sin inventar una corrección con un único punto.
7. Registra una condición en la que la lectura no deba usarse para decidir.

Registro mínimo

Referencia / cm	**Medida / cm**	**Estado / observación**
Posición 1	Por obtener	Objeto y montaje
Posición 2	Por obtener	Misma referencia
Sin eco	Sin distancia válida	Tiempo de espera

La diferencia entre referencia y medida se describe primero en centímetros. Un porcentaje puede resultar confuso cerca de cero. El propósito del ejercicio es encontrar si la conversión tiene sentido, si se conserva una referencia y si el programa trata la ausencia de eco. La calibración detallada de un sistema ultrasónico completo queda fuera de esta primera práctica.

Aplicación y límites de una medida de proximidad

Clase 07 · Distancia: tiempo de vuelo, adaptación de nivel y lecturas inválidas · 6/6

Una aplicación de apoyo a drones puede usar distancia para demostrar principios de proximidad o para un banco de exploración. La lectura de un módulo educativo no basta para prometer navegación o prevención de colisiones. La clase enseña a observar cobertura, tiempos y casos inválidos. Esa forma de trabajar permite después comparar otras tecnologías de sensado sin creer que todas fallan de la misma manera.

Dato repetido

La pantalla repite 25 cm durante un fallo. ¿Qué necesita saber la aplicación?

Cómo lo explicaríamos Si se obtuvo una lectura nueva, cuándo ocurrió y qué estado devolvió la adquisición. Repetir un valor almacenado no demuestra que el objeto permanezca a 25 cm.

Investiga por qué una superficie inclinada puede cambiar el eco y por qué la velocidad del sonido no es una constante independiente del ambiente. Explica la consecuencia práctica sin resolver un modelo acústico avanzado. El reto de transferencia consiste en escribir una política para dato inválido: aviso, conservación con edad o suspensión de una decisión que dependía de esa lectura.

Entrega de la clase

Conversión explicada, tabla de contraste y tratamiento de lectura inválida.

Fuentes para ampliar

[MECRO] MECRO. Laboratorio Electrónica con ESP32: veinte proyectos.

[HC] Wokwi. Referencia del sensor HC-SR04.

[GPIO] Espressif Systems. GPIO y restricciones de ESP32 clásico.

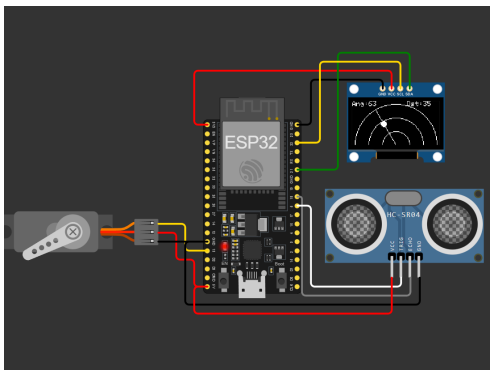
[WOKWI] Wokwi. ESP32 Simulation: periféricos y límites.

Tres cosas diferentes: mando, posición y medida

Clase 08 · Taller 1: servomotor y explorador ultrasónico · 1/6

Al terminar Coordinar una orden de posición con una lectura de distancia y conservar un registro coherente.

El proyecto de radar del laboratorio coordina un servo con un sensor ultrasónico. El nombre sirve para presentar el explorador, pero la medición es acústica. El programa puede ordenar un ángulo y dibujar una línea; eso no demuestra que el mecanismo ya alcanzó esa posición cuando se adquirió el dato. Nuestra primera tarea de integración es reconocer esa diferencia y guardar qué ocurrió en cada etapa.



MECRO, proyecto 11. Esquema original del laboratorio para reconocer la arquitectura en Wokwi. Para montaje físico se aplica la tabla y las correcciones de esta lectura.

En una carga de sensor, la orientación del soporte forma parte del significado de la medida. Un registro útil puede contener orden, tiempo de orden, tiempo de adquisición, distancia y estado. Si no hay un sensor de posición independiente, se identifica la posición como ordenada o estimada. Esta precisión del lenguaje ayuda a tomar decisiones de montaje sin exigir instrumentación avanzada.

Ancho de pulso y respuesta del servo

Clase 08 · Taller 1: servomotor y explorador ultrasónico · 2/6

$$T = \frac{1}{50} = 20\,\text{ms}, \qquad D = \frac{1.5}{20} = 7.5\,\%$$

El pulso de 1,5 ms es un ejemplo habitual de referencia central; el recorrido efectivo se verifica en el servo y el mecanismo reales.

Un servo convencional interpreta el ancho de un pulso repetido según su interfaz. Los límites y el sentido dependen del modelo y del montaje. No se enseña que cualquier servo tenga exactamente 0° a 180° entre 1 y 2 ms. En la práctica se comienza en una posición sin interferencia y con un intervalo permitido por su documentación. El archivo c08_servo usa un margen de ejercicio que el docente debe contrastar con el dispositivo.

Una orden y lo que falta conocer

Dato	Significado	Comprobación
Pulso / μs	Mando eléctrico	Señal observada
Ángulo solicitado	Convención del programa	Calibración del montaje
Posición real	Estado del mecanismo	Observación o sensor

La cuenta de duty depende de la resolución configurada. Con 16 bits y periodo de 20000 μs, un pulso de 1500 μs corresponde aproximadamente a 4915 cuentas. No se copia ese código a una configuración de 8 bits. La misma fracción exige otro número porque cambió la representación digital.

Alimentación y montaje de banco

Clase 08 · Taller 1: servomotor y explorador ultrasónico · 3/6

El servo necesita energía para su motor interno y puede demandar corriente variable al arrancar o sostener carga. La señal del GPIO no aporta esa potencia. En el banco se prepara una fuente adecuada para el actuador y una referencia de señal compatible. El positivo de esa fuente no se une arbitrariamente con otras alimentaciones de la placa. El docente presenta el circuito completo antes de realizar movimientos.

Separación de funciones

Elemento	Trabajo	Qué se comprueba
GPIO13	Orden al servo	Nivel y pulso
Fuente del servo	Energía del actuador	Tensión y capacidad
GND de referencia	Interpretación de señal	Conexión definida
Soporte mecánico	Movimiento permitido	Sin interferencias

Reinicio al mover

El programa funciona quieto y la placa se reinicia cuando se mueve el servo. ¿Qué revisas antes de cambiar el algoritmo?

Cómo lo explicaríamos La alimentación, la demanda del actuador, las conexiones y la referencia del montaje. El síntoma puede pertenecer al suministro de energía; no se resuelve suponiendo que todo es un error de programación.

Cada equipo identifica el recorrido que puede observar y los puntos donde el soporte podría interferir. No se fuerza un extremo para “calibrar” una pieza. El propósito es producir órdenes repetibles dentro de un intervalo comprobado y conservar la condición de alimentación del ensayo.

Secuencia del explorador

Clase 08 · Taller 1: servomotor y explorador ultrasónico · 4/6

Tres momentos de una exploración

```
// Estado 1: ordenar una posicion permitida
commandServo(target);
// Estado 2: esperar la condicion de adquisicion
// Estado 3: adquirir y guardar el par coherente
record(target, sampleTime, distance, valid);
```

El fragmento describe una secuencia; las funciones representan tareas que el equipo debe relacionar con su proyecto. No se presenta como un firmware completo listo para compilar. Primero se prueba el servo con c08_servo y el sensor con c07_distancia. Después se integra un pequeño conjunto de posiciones. La condición para adquirir se define según el montaje y la evidencia disponible.

La práctica del brazo robótico sirve como variante de mando y filtrado del potenciómetro. Su señal se puede reutilizar para una base de cámara, pero no basta cambiar el dibujo del proyecto para conocer la orientación de una carga. El equipo explica qué variable mide realmente y qué movimiento solo ordena. Esta distinción se conservará cuando aparezcan motores con encoder.

Agenda de cuatro horas y evidencias

Clase 08 · Taller 1: servomotor y explorador ultrasónico · 5/6

Taller presencial 1

Minutos	Trabajo	Producto
0-25	Revisión de esquemas	Mapa del equipo
25-70	Servo y señales	Intervalo comprobado
70-120	Ultrasonido y nivel	Referencia de distancia
120-175	Integración por estados	Registro coherente
175-215	Pruebas y diagnóstico	Fallo explicado
215-240	Demostración y cierre	Entrega del equipo

1. Comprueba por separado cada subsistema antes de integrar.
2. Ordena tres posiciones permitidas y registra qué has observado.
3. Relaciona cada distancia con la orden y el instante que le corresponden.
4. Trata un caso sin eco y conserva el estado en lugar de inventar distancia.
5. Intercambia roles de cableado, programación, medida y registro.
6. Entrega una conclusión sobre alimentación y otra sobre sincronización.

Los alumnos con kit trabajan con su material confirmado. Los demás usan el simulador y los bancos compartidos programados para el taller. En el registro se identifica qué comprobación se realizó físicamente y cuál pertenece a una ejecución de simulación. El resultado común del curso es la explicación del sistema y de su evidencia, con el modo de trabajo documentado.

Qué se acepta al cerrar el taller

Clase 08 · Taller 1: servomotor y explorador ultrasónico · 6/6

La entrega debe permitir que otra persona reconstruya el ensayo: esquema vigente, configuración de señal, intervalo permitido, código identificado y una tabla de posiciones y lecturas. Una animación atractiva no sustituye ese registro. Tampoco se exige una exploración de muchos puntos antes de resolver tres casos coherentes. La calidad se reconoce en que el equipo puede explicar de dónde salió cada dato y qué falta comprobar.

Misma figura, distinto registro

Dos equipos dibujan el mismo barrido. Uno guarda solo distancias y el otro guarda orden, instante y estado. ¿Cuál permite investigar una lectura fuera de secuencia?

Cómo lo explicaríamos El segundo conserva las referencias necesarias. La figura por sí sola no permite saber cuándo se adquirió cada dato.

Para investigar, compara el servo con el stepper del proyecto 13: señal de mando, energía, estado que se conoce y mecanismo de transmisión. Deja el stepper como ampliación hasta disponer de motor y driver coherentes. La siguiente parte del curso profundiza en potencia, medición de movimiento y realimentación, usando un banco DC de baja tensión.

Entrega de la clase

Esquema revisado, tres registros coherentes y diagnóstico de una condición de fallo.

Fuentes para ampliar

[MECRO] MECRO. Laboratorio Electrónica con ESP32: veinte proyectos.

[SERVO] Wokwi. Referencia del servomotor.

[HC] Wokwi. Referencia del sensor HC-SR04.

[BOARD] Espressif Systems. ESP32-DevKitC V4: guía de la placa.

[PWM] Espressif Systems. Arduino-ESP32: LEDC.

Una cadena con funciones separadas

Clase 09 · Motores y etapa de potencia: la orden no es la energía · 1/6

Al terminar Reconocer un driver y caracterizar un motor DC de banco sin confundir PWM con velocidad medida.

El ESP32 produce señales de mando; la etapa de potencia dirige energía hacia la carga. Un motor DC puede consumir corrientes muy superiores a las de un GPIO y generar perturbaciones al conmutar o cambiar de movimiento. Por eso no se conecta como si fuera otro LED. El banco del curso identifica fuente, driver, motor, referencia y señal. Cada elemento aparece en el esquema antes de ejecutar el programa.

Cadena del banco DC

Elemento	Función	Evidencia que se busca
ESP32	Generar la orden	Pin y configuración
Driver	Conmutar energía	Referencia y conexiones
Fuente	Aportar potencia	Condición de alimentación
Motor	Producir movimiento	Estado y respuesta

El proyecto 15 del laboratorio aporta una base de control de dirección y PWM con L293D, pero al revisar la página su enlace de simulación está vacío y su imagen no se encuentra en la ruta publicada. El curso no depende de ese recurso ausente: aporta un ejemplo nuevo de mando de banco, una tabla de interfaz y una práctica virtual numérica. El montaje con motor se organiza con el material confirmado o en banco compartido.

Un motor DC de escobillas no representa toda la electrónica de un motor BLDC de dron. El primero sirve aquí para ver la separación entre orden y potencia y para aprender medición y realimentación. El ESC de un BLDC cumple otra interfaz y controla sus fases. Se compara la arquitectura sin trasladar automáticamente conexiones ni parámetros de un dispositivo al otro.

Puente H, habilitación y alimentación

Clase 09 · Motores y etapa de potencia: la orden no es la energía · 2/6

Un puente H permite aplicar distintas condiciones a los extremos de un motor. Las entradas de dirección y la habilitación deben interpretarse con la hoja del driver concreto. Deshabilitar, dejar libre y frenar no son necesariamente el mismo estado eléctrico. La clase comienza con movimiento en un solo sentido y parada de mando, de manera que la observación sea fácil de relacionar con las señales.

Perfil de conexiones del ejemplo nuevo

Señal	GPIO clásico	Papel
ENABLE	25	PWM de mando
IN1	26	Dirección del banco
IN2	27	Dirección del banco
GND	Referencia	Según esquema del driver

En el L293D se distingue la alimentación lógica VCC1 de la de potencia VCC2. La hoja de TI especifica condiciones de operación; no respalda alimentar VCC1 con 3,3 V como propone una explicación del sitio. El banco utiliza la alimentación lógica que exige el componente y revisa por separado la compatibilidad de sus entradas con el ESP32. Separar alimentaciones de señal y potencia no constituye por sí solo aislamiento galvánico.

Una tensión que desaparece

La fuente entrega una tensión, pero en los bornes del motor se observa menos durante el movimiento. ¿La diferencia demuestra que el ADC falla?

Cómo lo explicaríamos No. Primero se identifica qué se midió y se considera la caída de tensión en la etapa de potencia y en las conexiones bajo esa carga. El circuito se estudia antes de atribuir el síntoma a otro subsistema.

Caracterizar antes de controlar

Clase 09 · Motores y etapa de potencia: la orden no es la energía · 3/6

La misma orden PWM no garantiza la misma velocidad con distintas cargas, tensiones o condiciones de arranque. Puede existir una zona en la que el motor aún no inicia movimiento y otra en la que sí continúa girando después de haber arrancado. Para estudiar esa respuesta se conservan fuente, montaje y sentido. Primero registramos el efecto observado; la próxima clase añadirá un método explícito de medición de RPM.

Datos sintéticos para practicar lectura

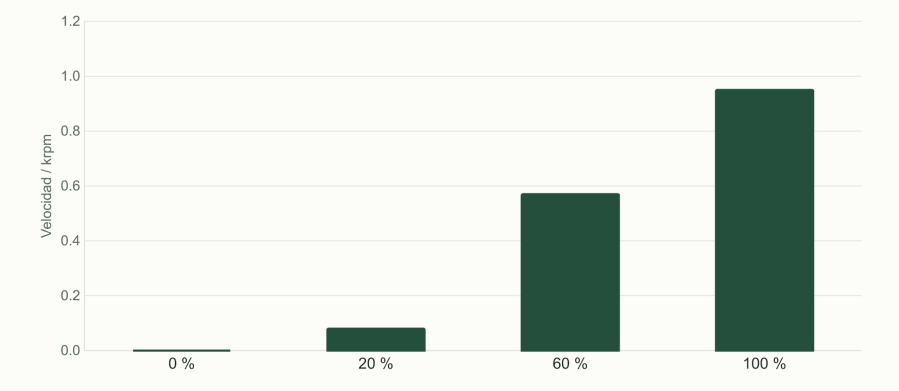
Mando / %	Velocidad / RPM	Condición
0	0	Modelo didáctico
20	80	Modelo didáctico
60	570	Modelo didáctico
100	950	Modelo didáctico

Mando y velocidad

En el caso sintético, pasar de 20 % a 60 % cambia de 80 a 570 RPM. ¿Puedes anunciar una ley universal PWM-RPM?

Cómo lo explicaríamos No. La tabla describe el caso declarado y muestra que mando y respuesta son magnitudes diferentes. Para un motor físico se requiere medir bajo condiciones identificadas.

La denominación “control de velocidad” se utiliza a veces para mover un potenciómetro que cambia PWM. En esta fase el programa controla el mando, no verifica velocidad. Ese lenguaje se precisa al cerrar el lazo. La distinción será visible cuando aparezca una perturbación: el potenciómetro puede conservar su posición mientras la velocidad cambia.



Datos sintéticos del caso de clase. El banco real debe caracterizarse con su fuente, motor, driver y carga.

Mando explícito y estado inicial

Clase 09 · Motores y etapa de potencia: la orden no es la energía · 4/6

Núcleo del banco c09_motor

```
// El mando inicia en cero.
if (command == '+') duty = min(160, duty+20);
if (command == 's') duty = 0;
if (uint32_t(now-commandAt) > 2000) duty = 0;
motor.write(duty);
```

El ejemplo completo inicia con el driver deshabilitado y acepta órdenes por el monitor serie. El mando de aula se limita y vence si no se renueva. Ese comportamiento se documenta para evitar interpretar una parada temporal como un fallo del driver. La versión incluida trabaja en un solo sentido: el alumno estudia la tabla del componente antes de ampliar una inversión.

La frecuencia de PWM es una elección de diseño del banco. No se enseña que 5 kHz sea una frecuencia "ideal" universal para L293D. El efecto audible, las pérdidas de conmutación y el comportamiento del conjunto pueden cambiar con la frecuencia. El docente elige una condición de referencia compatible con su montaje y la mantiene durante la primera comparación. Cambiar frecuencia y carga simultáneamente no produce una prueba fácil de interpretar.

Laboratorio de mando y respuesta

Clase 09 · Motores y etapa de potencia: la orden no es la energía · 5/6

1. Presenta el esquema de la etapa y señala por dónde circula la energía del motor.
2. Comprueba alimentación lógica, alimentación de potencia e interfaz del componente.
3. Ejecuta c09_motor primero con la salida observada y sin una carga no revisada.
4. En el banco docente, ordena incrementos pequeños dentro del límite previsto y observa el inicio de movimiento.
5. Comprueba parada por orden y vencimiento del mando.
6. Registra configuración, orden, tensión observada y descripción del movimiento.
7. Prepara qué variable y qué sensor faltan para informar RPM.

La comparación se realiza con motor de baja tensión sujeto en el banco y bajo el procedimiento docente. El alumno sin motor puede estudiar la señal de habilitación y resolver los registros sintéticos; la evidencia del ensayo físico queda a cargo del equipo que lo ejecuta. El propósito común es explicar la cadena completa, no afirmar que un movimiento visto en pantalla valida corriente o temperatura del hardware.

Comparación con la arquitectura de drones

Clase 09 · Motores y etapa de potencia: la orden no es la energía · 6/6

Dispositivos que no se intercambian por nombre

Sistema	Interfaz de potencia	Qué se aprende aquí
DC de escobillas	Puente H o etapa adecuada	Mando y energía
Servo de posición	Control interno propio	Pulso y posición
BLDC con ESC	Conmutación electrónica	Cadena de interfaces

Para una carga mecánica auxiliar de UAV, elegir el actuador empieza por la función: posicionar un soporte, accionar una pieza o sostener una velocidad. El tipo de motor y la información que entrega su sistema cambian la arquitectura. En lugar de escoger por apariencia, el equipo escribe qué orden recibe y qué observación necesitaría para verificar su función.

Investiga en la hoja del driver el nombre y la condición de cada alimentación. Compara luego un estado deshabilitado con uno de frenado según su tabla funcional. Describe la diferencia sin generalizarla a todos los puentes H. El siguiente paso consiste en obtener una medida de movimiento cuya unidad y referencia podamos defender.

Entrega de la clase

Esquema de potencia, registro de mando y explicación de la medida que falta.

Fuentes para ampliar

[MECRO] MECRO. Laboratorio Electrónica con ESP32: veinte proyectos.

[L293] Texas Instruments. L293/L293D: hoja de datos SLRS008D.

[PWM] Espressif Systems. Arduino-ESP32: LEDC.

[MIT] Agarwal y Lang / MIT OpenCourseWare. 6.002 Circuits and Electronics; texto: Foundations of Analog and Digital Electronic Circuits (2005).

Contar no es todavía medir RPM

Clase 10 · Encoder e interrupciones: medir velocidad con una definición · 1/6

Al terminar Calcular RPM desde pulsos usando el conteo por vuelta y el tiempo realmente transcurrido.

Un encoder transforma movimiento en señales que el programa puede contar. El número de pulsos solo tiene significado de velocidad cuando sabemos cuánto movimiento representa y en qué tiempo ocurrió. Un sensor puede especificar pulsos por revolución del eje del motor, mientras el proyecto observa el eje de salida de una reducción. Además, una biblioteca puede contar un flanco, ambos flancos o transiciones de dos canales. La definición que importa es la cuenta efectiva por vuelta de la referencia observada.

Datos necesarios

Dato	Pregunta	Unidad
Cuenta efectiva por vuelta	¿Qué flancos cuenta?	cuentas/vuelta
Ventana real	¿Cuánto tiempo pasó?	s o ms
Referencia mecánica	¿Qué eje se observa?	Identificador
Conteo	¿Qué ocurrió en la ventana?	cuentas

El proyecto de motor de la web no incluye esta medición, por eso se añade como práctica nueva. Se puede comenzar con un generador de pulsos del banco o un modelo de señal. Después se relaciona con un encoder real identificado. No se asignan 20 cuentas por vuelta a cualquier motor: ese número es una constante de ejercicio que se sustituye al caracterizar el dispositivo.

Conversión con unidades

Clase 10 · Encoder e interrupciones: medir velocidad con una definición · 2/6

$$n_{rpm} = \frac{60\,\Delta N}{N_{vuelta}\,\Delta t_s} = \frac{60000\,\Delta N}{N_{vuelta}\,\Delta t_{ms}}$$

Nvuelta corresponde al método de conteo y al eje declarados.

Doce cuentas en una ventana

Se cuentan 12 flancos durante 100 ms. La referencia comprobada es 20 flancos por vuelta del eje observado. ¿Qué velocidad media representa?

Cómo lo explicaríamos 12/20 = 0,6 vueltas en 0,1 s. Son 6 vueltas por segundo y, por tanto, 360 RPM.

Si realmente transcurrieron 110 ms, usar 100 ms en la cuenta sobreestimaría la velocidad. El código conserva elapsed = now-last y utiliza esa duración. La ventana aporta una media durante un intervalo; no es una velocidad instantánea perfecta. En un cambio rápido, una ventana larga mezcla estados anteriores con el más reciente.

Una misma cuenta y dos referencias

Referencia	Duración	Resultado
20 cuentas/vuelta	100 ms, 12 cuentas	360 RPM
20 cuentas/vuelta	110 ms, 12 cuentas	327,3 RPM
40 cuentas/vuelta	100 ms, 12 cuentas	180 RPM

Resolución temporal y de conteo

Clase 10 · Encoder e interrupciones: medir velocidad con una definición · 3/6

$$\Delta n = \frac{60}{N_{vuelta}\, T_s}$$

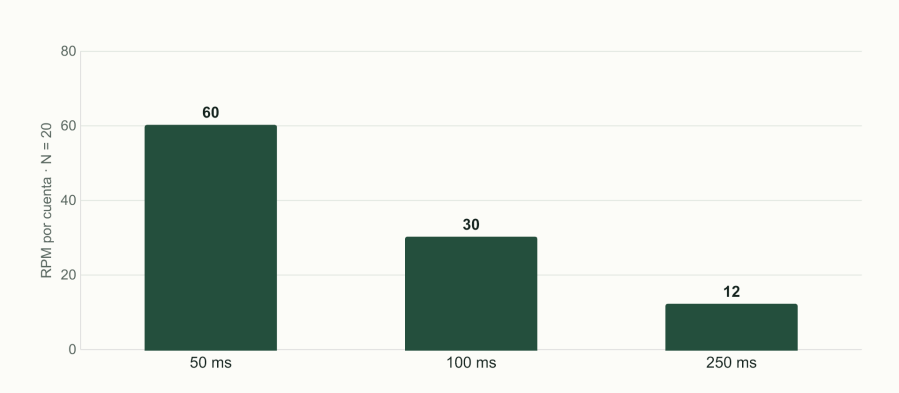
Una cuenta adicional cambia la estimación en esa cantidad de RPM para la ventana dada.

Con 20 cuentas por vuelta y ventana de 0,1 s, una sola cuenta representa 30 RPM. Un número con muchos decimales no añade información que el conteo no tiene. Una ventana de 0,25 s reduce ese salto a 12 RPM, pero tarda más en describir un cambio. El equipo compara esa compensación con el comportamiento que desea observar.

Caso didáctico, 20 cuentas/vuelta

Ventana	Salto por cuenta	Compensación
50 ms	60 RPM	Respuesta más frecuente
100 ms	30 RPM	Condición de referencia
250 ms	12 RPM	Promedio más prolongado

Un encoder de dos canales también puede permitir conocer dirección mediante la relación entre sus señales. La primera práctica cuenta solo flancos ascendentes de un canal para que su denominador sea transparente. La cuadratura se deja como ampliación después de verificar una vuelta y un método. Añadir canales sin revisar el factor de conteo puede multiplicar por dos o cuatro una lectura aparente.



Cálculo con veinte cuentas efectivas por vuelta. Ventana mayor mejora el paso de lectura y agrega espera.

Qué hace la interrupción y qué hace loop()

Clase 10 · Encoder e interrupciones: medir velocidad con una definición · 4/6

Reparto de tareas en c10_encoder

```
void IRAM_ATTR edge() {
  portENTER_CRITICAL_ISR(&mux);
  ++edges;
  portEXIT_CRITICAL_ISR(&mux);
}
// En loop(): copiar y poner a cero como una operacion.
portENTER_CRITICAL(&mux);
uint32_t count = edges;
edges = 0;
portEXIT_CRITICAL(&mux);
```

La rutina de interrupción realiza una tarea breve: conservar el evento de conteo. La conversión, impresión y decisión se hacen fuera. volatile evita ciertas suposiciones del compilador sobre accesos, pero no convierte una secuencia de copiar y poner a cero en una operación indivisible. Por eso el ejemplo protege ese intercambio de manera explícita.

La señal debe tener niveles compatibles y una forma que el sistema pueda interpretar. Un encoder con salida de colector abierto puede necesitar polarización; otro módulo puede entregar una señal que requiere adaptación. El banco identifica ese tipo de salida antes de conectar. Las transiciones espurias y el ruido no se corrigen inventando un nuevo número de cuentas por vuelta: se investiga la señal y el montaje.

Laboratorio de una vuelta y una ventana

Clase 10 · Encoder e interrupciones: medir velocidad con una definición · 5/6

1. Identifica el eje y la convención de conteo que utilizarás.
2. Antes de convertir a RPM, comprueba las cuentas de una vuelta observada o de una señal conocida.
3. Ejecuta c10_encoder con el generador de pulsos o el encoder preparado.
4. Calcula manualmente una fila del registro con elapsed real.
5. Compara ventanas de 50, 100 y 250 ms sin cambiar a la vez el método de conteo.
6. Busca un error de factor dos o cuatro en un caso de prueba del docente.
7. Conserva fuente de señal, condiciones y resultado con unidades.

El alumno sin encoder resuelve el conjunto de pulsos del curso y explica qué falta para trasladarlo a un motor real. En el taller compartido se contrasta el método con hardware disponible. Esta organización permite estudiar la medida antes de comprar un componente adicional y evita anunciar un encoder como parte del kit cuando su inventario aún no se ha confirmado.

Qué necesita el controlador de esta medición

Clase 10 · Encoder e interrupciones: medir velocidad con una definición · 6/6

El lazo de control recibirá una estimación de velocidad con su periodo, dispersión y posibilidad de fallo. Un valor cero puede corresponder a motor parado, una ventana sin pulsos o una señal ausente. La interpretación exige contexto. La aplicación conoce si ordenó movimiento, cuánto tiempo pasó y qué otras comprobaciones existen. Esos datos ayudan a formular un estado de diagnóstico, aunque no convierten cualquier ausencia de pulsos en una causa demostrada.

Precisión aparente

El monitor muestra 360,000 RPM con un salto de 30 RPM por cuenta. ¿Qué comunican esos tres decimales?

Cómo lo explicaríamos No demuestran una precisión milésima. El formato de impresión debe respetar la información y las limitaciones del método.

Investiga qué significa PPR o CPR en la ficha concreta de un encoder. Copia la definición, no solo el número. Si hay una reducción, señala a qué eje pertenece. Prepara una frase completa de tu método: "cuento flancos ascendentes de A, con tantas cuentas por vuelta del eje de salida, durante el intervalo registrado". Esa frase será parte de la entrega del lazo cerrado.

Entrega de la clase

Conversión de una fila, definición de cuenta por vuelta y comparación de ventanas.

Fuentes para ampliar

[MECRO] MECRO. Laboratorio Electrónica con ESP32: veinte proyectos.

[GPIO] Espressif Systems. GPIO y restricciones de ESP32 clásico.

[UART] Espressif Systems. Arduino-ESP32: Serial/UART.

[FBS] Åström y Murray. Feedback Systems: An Introduction for Scientists and Engineers; edición electrónica de los autores.

La pregunta que abre el lazo cerrado

Clase 11 · Control realimentado: P, PI y una introducción a PID · 1/6

Al terminar Comparar respuestas ante una perturbación y explicar el papel de P, I y D con tiempo de muestreo y saturación.

En lazo abierto se fija un mando y se observa qué ocurre. En lazo cerrado se mide una salida, se compara con una referencia y se actualiza el mando usando esa diferencia. No se vuelve perfecto por añadir la palabra PID. El controlador trabaja con un sensor, un actuador, retardos y límites. Si la referencia es imposible para el conjunto, el algoritmo no puede crear capacidad física que el actuador no tiene.

Elementos del banco

Elemento	Ejemplo	Qué representa
Referencia r	500 RPM	Objetivo declarado
Medida y	RPM del encoder	Salida observada
Error e	r menos y	Diferencia de seguimiento
Mando u	Código PWM	Acción sobre el banco

La práctica nueva usa primero una planta matemática de aula y luego el banco físico disponible. El modelo es de primer orden y declara su ganancia y su constante de tiempo. Sus respuestas no se presentan como medidas del kit MECRO. La ventaja didáctica consiste en poder introducir una perturbación conocida y repetir exactamente las condiciones.

Qué añade cada acción

Clase 11 · Control realimentado: P, PI y una introducción a PID · 2/6

$$e_k = r_k - y_k, \qquad u_P = K_p e_k, \qquad I_k = I_{k-1} + K_i e_k \Delta t$$

Las ganancias tienen unidades que dependen de referencia, medida y mando. Conservar Δt evita cambiar el controlador sin querer al cambiar el periodo.

$$D_k = -K_d \frac{y_k - y_{k-1}}{\Delta t}, \qquad u = \text{limitar}(u_{base} + u_P + I_k + D_k)$$

Se muestra D sobre la medida como una convención posible. La implementación completa debe declarar su elección y su tratamiento de ruido.

Acciones y observaciones

Acción	Qué relaciona	Qué hay que vigilar
P	Error actual y mando	Error persistente y ganancia
I	Acumulación temporal	Acumulación en saturación
D	Cambio de la medida	Ruido y diferencia temporal

En un banco de velocidad puede bastar PI para el objetivo introductorio. PID nombra una familia, no la obligación de usar tres términos distintos de cero. La derivada se estudia con un caso de ruido para comprender su coste. El ajuste se hace sobre una configuración identificada y se conserva una curva por cambio.

Un modelo transparente para comparar

Clase 11 · Control realimentado: P, PI y una introducción a PID · 3/6

$$\dot{y} = \frac{\text{máx}(0, 4u - L) - y}{0.35}$$

Modelo sintético: y en RPM, u en cuentas de 0 a 255 y L como perturbación equivalente en RPM. No representa un motor identificado.

La referencia del ejercicio pasa de cero a 500 RPM. Después se aplica una perturbación equivalente de 100 RPM. El mando base es r/4. La comparación incluye lazo abierto, P con ese mando base y PI. Como las entradas del modelo se conservan, una diferencia entre respuestas puede relacionarse con el controlador elegido. Las curvas y el CSV del curso se generan a partir de estas ecuaciones con paso declarado.

Una cuenta de controlador

Si el error es 50 RPM, Kp vale 0,12 cuentas/RPM, Ki vale 0,6 cuentas/(RPM·s) y Δt = 0,02 s, ¿qué contribuciones obtienes?

Cómo lo explicaríamos P aporta 6 cuentas. El incremento de I es 0,6 × 50 × 0,02 = 0,6 cuentas en ese paso. El acumulado I depende de los pasos anteriores.

Sin integral, el modelo con mando base y P necesita un error persistente para compensar la perturbación. En régimen del caso, e = 100/(1 + 4 × 0,12), aproximadamente 67,6 RPM. La integral puede aportar la acción mantenida que reduce ese error si el sistema tiene capacidad y el ajuste resulta adecuado. La conclusión pertenece a este modelo y a esas condiciones.

Saturación y acumulación integral

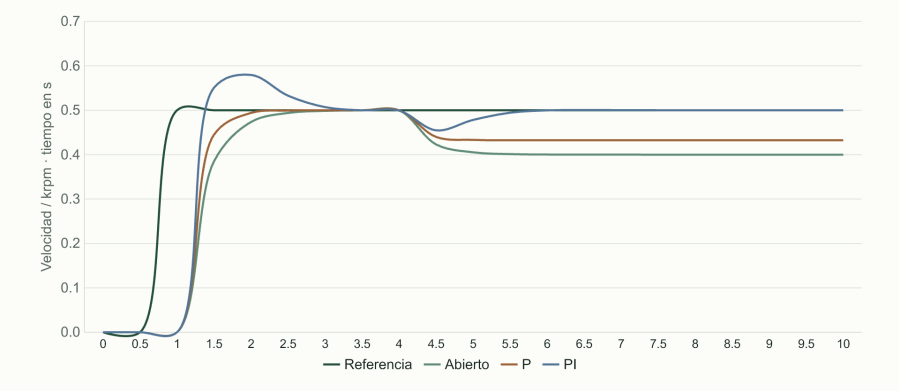
Clase 11 · Control realimentado: P, PI y una introducción a PID · 4/6

Acumulación condicionada del ejemplo

```
float trial = integral + Ki * error * dt;
float unsat = base + Kp * error + trial;
bool inside = unsat >= 0 && unsat <= 255;
bool returning = (unsat > 255 && error < 0)
               || (unsat < 0 && error > 0);
if (inside || returning) integral = trial;
float u = constrain(base + Kp*error + integral,
                    0.0f, 255.0f);
```

El mando tiene un intervalo. Si se sigue acumulando integral mientras el actuador ya está en un extremo y el error exige avanzar todavía más, puede demorarse la recuperación al cambiar la situación. El fragmento conserva el incremento cuando la salida está dentro o cuando ayuda a regresar del límite. Es una estrategia introductoria que debe estudiarse con su definición exacta; no se llama anti-windup a cualquier recorte sin explicar qué ocurre con el estado integral.

El firmware c11_pi_virtual ejecuta una planta de paso fijo y produce CSV. Ese tiempo pertenece al modelo. El banco real mide intervalos y respuesta del conjunto. Al pasar de uno a otro no se copian ganancias como garantía de funcionamiento. Se vuelve a identificar escala, límites, dirección de acción y condiciones de la medida.



Modelo sintético: $\tau = 0{,}35$ s, ganancia 4 RPM/cuenta, dt = 20 ms; carga de 100 RPM equivalentes desde 4 s.

Laboratorio de comparación con una perturbación

Clase 11 · Control realimentado: P, PI y una introducción a PID · 5/6

1. Dibuja referencia, controlador, planta, medida y perturbación.
2. Abre el CSV sintético y localiza el escalón de referencia y el cambio de carga.
3. Compara lazo abierto y P conservando el mando base y las mismas condiciones.
4. Compara P y PI; describe error persistente y evolución temporal.
5. Introduce un caso de saturación y observa el estado integral.
6. Estudia D con una medida que contiene una variación rápida; explica por qué puede amplificarla.
7. En el banco docente disponible, repite primero una prueba pequeña con referencia alcanzable.

Qué comparar en una respuesta

Pregunta	Dato que la apoya
¿Sigue la referencia?	Error y evolución
¿Compensa la perturbación?	Estado antes y después
¿Satura?	Mando y límites
¿El cambio es repetible?	Condiciones conservadas

Las curvas se leen con ejes y unidades. Un cruce de la referencia puede ser sobrepaso; una respuesta rápida no es necesariamente la mejor si produce oscilación o exige mando saturado. El alumno escribe una conclusión que incluya una ventaja y una condición observada del caso, sin convertir una única cifra en un ranking universal.

Relación con drones y siguiente paso

Clase 11 · Control realimentado: P, PI y una introducción a PID · 6/6

En un UAV puede haber varios lazos y estimaciones entre el mando humano, la orientación y la actuación. El curso no los condensa en una receta de tres ganancias. Enseña las piezas conceptuales que permiten hacer mejores preguntas: qué variable se controla, qué sensor la informa, a qué frecuencia se actualiza y qué límites tiene la acción. El proyecto 17 se revisará como demostración de mando y control proporcional de banco, con su alcance corregido.

Un número llamado pid

Un programa calcula error × kp y guarda el resultado en pid_roll. ¿Qué acción implementa?

Cómo lo explicaríamos Una acción proporcional. El nombre de la variable no añade integral ni derivada.

Para ampliar, consulta Feedback Systems de Åström y Murray en sus secciones de realimentación y PID. Lee el esquema de bloques junto a un caso del banco. La meta de esta primera aproximación es explicar por qué se produjo una respuesta y qué dato permitiría revisar una hipótesis. Las ganancias y las curvas forman parte de esa explicación, no un sustituto de ella.

Entrega de la clase

Gráficas identificadas, caso P/I resuelto y comparación de dos respuestas.

Fuentes para ampliar

[MECRO] MECRO. Laboratorio Electrónica con ESP32: veinte proyectos.

[FBS] Åström y Murray. Feedback Systems: An Introduction for Scientists and Engineers; edición electrónica de los autores.

[PWM] Espressif Systems. Arduino-ESP32: LEDC.

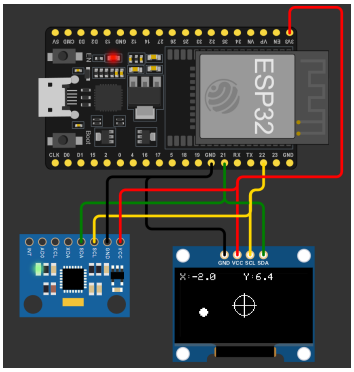
[UART] Espressif Systems. Arduino-ESP32: Serial/UART.

Qué significa seis ejes

Clase 12 · IMU y orientación de banco: medir antes de estabilizar · 1/6

Al terminar Distinguir aceleración, velocidad angular e inclinación estimada y obtener un registro con unidades.

La IMU del proyecto MPU6050 combina tres componentes de aceleración y tres de velocidad angular. Esos seis canales no equivalen a seis ángulos medidos directamente. El acelerómetro y el giróscopo responden a magnitudes distintas. Antes de interpretar un número se necesita un sistema de ejes ligado al montaje, una unidad y una configuración de rango.



MECRO, proyecto 16. Esquema original del laboratorio para reconocer la arquitectura en Wokwi. Para montaje físico se aplica la tabla y las correcciones de esta lectura.

En reposo, la lectura del acelerómetro permite observar la dirección asociada a gravedad bajo las condiciones del banco. Durante movimiento se añaden otros efectos y esa lectura no se interpreta automáticamente como inclinación. El giróscopo aporta velocidad angular; al integrarla se obtiene una estimación de cambio angular que también acumula error por sesgo. Ambas observaciones pueden complementarse, pero ninguna se vuelve una orientación completa por cambiar el nombre de una variable.

La clase separa los atributos del simulador de las unidades que devuelve una biblioteca. En Wokwi se pueden introducir valores en g y grados por segundo. La biblioteca Adafruit utilizada presenta aceleración en m/s^2 y giro en rad/s. Ese cambio de representación debe reconocerse antes de aplicar fórmulas o escalas.

Un ángulo a partir de la gravedad del banco

Clase 12 · IMU y orientación de banco: medir antes de estabilizar · 2/6

$$\phi = \operatorname{atan2}(a_y, a_z), \qquad \theta = \operatorname{atan2}\left(-a_x, \sqrt{a_y^2 + a_z^2}\right)$$

Convención del ejemplo: los resultados de atan2 están en radianes; se convierten a grados si se van a presentar así. La orientación de ejes y el signo se comprueban en el montaje.

Estas expresiones se usan en la práctica cuasiestática, con la referencia de ejes declarada. atan2 considera la relación entre componentes y su cuadrante. Multiplicar aceleración por una constante no representa en general esa geometría. Cerca de una inclinación pequeña puede parecer que dos curvas se parecen, pero la coincidencia local no demuestra una conversión universal.

Inclinación de 30 grados

En un caso ideal estático, ay = 4,903 m/s^2 y az = 8,493 m/s^2. ¿Qué informa atan2(ay,az) en grados?

Cómo lo explicaríamos Aproximadamente 30°. Multiplicar ay por 5 produciría 24,5 sin una justificación general de unidades ni de geometría.

Magnitud y unidad de la biblioteca

Variable	Unidad	Interpretación
a.acceleration.x	m/s^2	Componente del acelerómetro
g.gyro.x	rad/s	Velocidad angular
Ángulo calculado	rad o grados	Estimación bajo hipótesis

Sesgo, ruido y configuración

Clase 12 · IMU y orientación de banco: medir antes de estabilizar · 3/6

El sesgo es una desviación sistemática de la lectura. Una lectura con variación pequeña puede conservar un sesgo grande: repetir bien no equivale a medir correctamente. En un ensayo de reposo se observan muestras y se explica qué referencia permite interpretar cada eje. Para el giróscopo, una media distinta de cero en un montaje realmente quieto puede orientar una estimación de sesgo bajo esas condiciones.

Tres comprobaciones iniciales

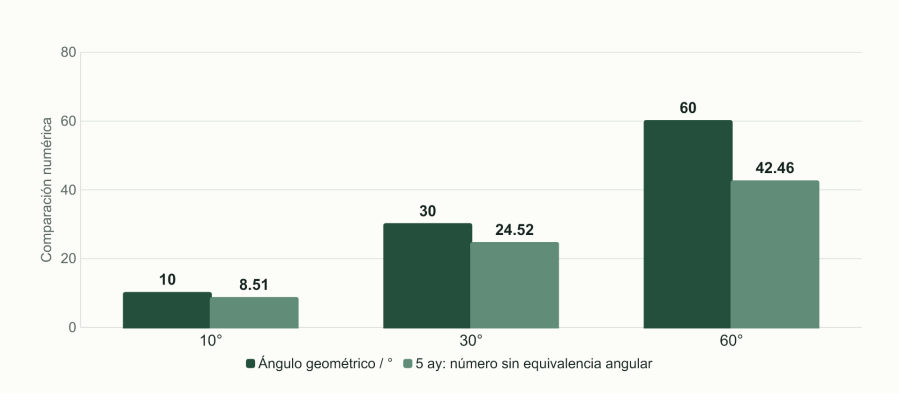
Prueba	Qué observar	Qué no concluye sola
Reposo	Norma y giro medio	Exactitud completa
Inclinación lenta	Signo y orientación	Actitud dinámica de vuelo
Movimiento rápido	Cambios y dispersión	Separación perfecta de efectos

El rango configurado y el filtrado del sensor influyen en lo que se obtiene. Un rango mayor permite representar magnitudes mayores, pero no significa más detalle para todos los cambios pequeños. Un filtro puede reducir componentes rápidas y añadir retardo. El docente conserva el rango y el filtro al comparar dos posiciones, y modifica una elección a la vez cuando estudia su efecto.

Dato estable, explicación incorrecta

El nivel de burbuja parece suave al mover el equipo rápidamente. ¿La suavidad demuestra que separó inclinación y aceleración dinámica?

Cómo lo explicaríamos No. Puede haber filtrado o una representación gráfica suavizada. Hace falta relacionar hipótesis, movimiento y medida.



Caso estático ideal: ay = g sen(roll). atan2(ay, az) recupera el ángulo bajo las hipótesis de la lectura.

Lectura del firmware de IMU

Clase 12 · IMU y orientación de banco: medir antes de estabilizar · 4/6

Fragmento de c12_imu

```
sensors_event_t a, g, temp;
imu.getEvent(&a, &g, &temp);
float roll = atan2f(a.acceleration.y,
                    a.acceleration.z) * 180.0f/PI;
float gx_rad_s = g.gyro.x;
```

El archivo completo prepara I2C21/22, verifica inicialización, configura rangos y registra tiempo con los valores. El alumno identifica cuál de sus variables es una medida y cuál es una transformación. Cambiar la orientación física del módulo puede invertir signos o intercambiar ejes; la corrección se documenta como una convención del montaje, no como un ajuste secreto para que una pantalla parezca correcta.

El proyecto de mouse aéreo aporta una aplicación de velocidad angular. Antes de probar BLE se puede observar cómo cambia gx_rad_s en serie. La radio Bluetooth no está implementada en el simulador ESP32 de Wokwi consultado; la experiencia de mouse requiere hardware y una biblioteca compatible. La existencia del enlace del proyecto no garantiza que ese perfil inalámbrico pueda demostrarse dentro de la simulación.

Laboratorio de orientación lenta

Clase 12 · IMU y orientación de banco: medir antes de estabilizar · 5/6

1. Identifica ejes, alimentación y conexiones del módulo utilizado.
2. En el simulador, reconoce las unidades de los atributos y compáralas con la biblioteca.
3. Ejecuta c12_imu y conserva un registro de reposo.
4. Realiza inclinaciones lentas de referencia y comprueba signo y tendencia.
5. Calcula un caso a mano con atan2 y conserva la convención.
6. Muestra un movimiento dinámico y explica por qué la interpretación cambia.
7. Distingue validez de la comunicación de validez de una estimación angular.

El modelo de Wokwi permite explorar componentes y verificar conversiones conocidas. En el banco físico se describen el movimiento realizado y la referencia disponible. Una regla o soporte angular puede ayudar a contrastar un caso lento. No se inventa una medida de inclinación exacta cuando el método solo permitió observar el sentido y una variación aproximada.

Por qué este bloque precede al mando

Clase 12 · IMU y orientación de banco: medir antes de estabilizar · 6/6

El controlador de la clase siguiente comparará una referencia y una estimación. Si ambas usan escalas o ejes distintos, el error numérico puede tener un significado falso aunque el programa compile. Por eso primero se aprende a leer la IMU. El estudiante debe poder señalar de dónde salió cada número antes de proponer una acción correctiva.

$$\|a\| = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

Comparar la norma con una referencia gravitatoria en un ensayo de banco aporta contexto; no constituye por sí solo un detector universal de actitud válida.

Investiga cómo se acumula un sesgo pequeño del giróscopo al integrar durante varios segundos. Como caso ideal, 0,5 grados/s sostenidos durante 10 s producen 5 grados de cambio estimado aunque el equipo no haya girado. Relaciona ese cálculo con la necesidad de referencias y calibración. El curso introduce el problema sin convertirlo en una implementación completa de estimación de vuelo.

Entrega de la clase

Registro de reposo e inclinación, caso atan2 y convención de unidades.

Fuentes para ampliar

[MECRO] MECRO. Laboratorio Electrónica con ESP32: veinte proyectos.

[IMU] Adafruit. MPU6050: biblioteca, cableado y lectura.

[IMUSIM] Wokwi. MPU6050: atributos del modelo.

[WIRE] Espressif Systems. Arduino-ESP32: I2C.

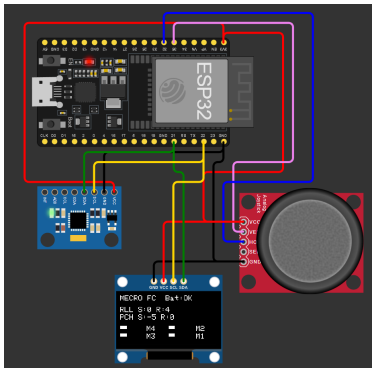
[WOKWI] Wokwi. ESP32 Simulation: periféricos y límites.

Qué hace el proyecto y cómo lo vamos a usar

Clase 13 · Mando y control de banco: interpretar el proyecto de vuelo · 1/6

Al terminar Transformar un joystick a una referencia normalizada y relacionarla con una medida y una acción proporcional declaradas.

El proyecto 17 del laboratorio lee joystick e IMU, calcula una corrección proporcional y muestra números en serie. Es una buena oportunidad para conectar varias ideas del curso, siempre que su nombre y su alcance se expliquen con precisión. La edición docente lo presenta como “Mando e IMU: demostración proporcional de banco”. Las variables impresas como salidas no son una prueba de actuación sobre motores ni una validación de vuelo.



MECRO, proyecto 17. Esquema original del laboratorio para reconocer la arquitectura en Wokwi. Para montaje físico se aplica la tabla y las correcciones de esta lectura.

Primero se separan tres bloques. El mando humano produce una referencia. La medición y su procesamiento producen una estimación. La comparación genera un error con unidad. Después se decide qué acción ilustrativa mostrar o aplicar al banco preparado. Si se mezclan esos bloques en una sola lista de números llamada “controlador”, cuesta detectar si el error proviene del joystick, del sensor, de la escala o del signo.

La conexión con drones es conceptual y tangible a la vez: el alumno sostiene el mando y el módulo de sensores, observa sus cambios y explica cómo podría organizarse esa cadena en un sistema más completo. Mantener un propósito de banco permite ensayar hipótesis sin atribuir al ejercicio funciones que no se han desarrollado.

Centro, escala y zona muerta del joystick

Clase 13 · Mando y control de banco: interpretar el proyecto de vuelo · 2/6

Un joystick analógico no siempre entrega exactamente 2048 al soltarlo ni recorre de forma simétrica hasta ambos extremos. Se registran centro y extremos útiles de cada eje. La normalización transforma esos valores a una convención, por ejemplo de -1 a 1. Una zona muerta conserva cero ante pequeñas variaciones alrededor del centro. Su tamaño se justifica con el comportamiento observado, no con el deseo de ocultar un montaje inestable.

$$x_n = \begin{cases} \frac{x-c}{x_{max}-c}, & x \geq c \\ \frac{x-c}{c-x_{min}}, & x < c \end{cases}$$

Se comprueba que los denominadores sean válidos y se limita el resultado al intervalo de mando previsto.

Datos para una calibración

Dato	Qué significa	Qué registrar
Centro c	Reposo observado	Valor y dispersión
Extremos	Recorrido útil	Mínimo y máximo
Zona muerta	Cambio ignorado	Criterio y amplitud
Signo	Sentido de la referencia	Convención de ejes

Centro desplazado

Un eje queda en 2150 y la escala fija espera 2048. ¿El mando de referencia será necesariamente cero?

Cómo lo explicaríamos No. La calibración o la zona muerta deben representar el centro real y su criterio. No se arregla cambiando el sensor de orientación, porque el origen de esta diferencia está en el mando.

Una referencia y una medida comparables

Clase 13 · Mando y control de banco: interpretar el proyecto de vuelo · 3/6

Para calcular un error angular, referencia y estimación deben compartir unidad y convención. El proyecto original usa una multiplicación de aceleración como aproximación angular. En el curso se sustituye por la estimación cuasiestática explicada en la clase 12 o por un caso numérico de referencia identificado. El alumno señala el punto donde una medida pasa a ser una estimación y qué hipótesis permite interpretarla.

$$e_\phi = \phi_{ref} - \widehat{\phi}, \qquad u_P = K_p e_\phi$$

La ganancia relaciona unidades de error y de salida. Aquí se observa una acción proporcional de banco.

Signo de la acción

La referencia del caso es 10° y la estimación es 6°. Con Kp = 2 unidades/°, ¿qué acción proporcional resulta?

Cómo lo explicaríamos El error es 4° y la acción resulta 8 unidades del mando definido. El signo físico que corrige la desviación depende de la interfaz y del montaje; se comprueba antes de cerrar un lazo real.

Un cambio de signo puede producir realimentación positiva si la acción agranda la desviación en lugar de reducirla. Por eso la primera comprobación del banco es pequeña y observable. Un nombre como “roll” no determina por sí solo qué dirección representa el eje. La convención queda dibujada o anotada junto a la pieza.

Ordenar el programa por bloques

Clase 13 · Mando y control de banco: interpretar el proyecto de vuelo · 4/6

Flujo de una iteración de banco

```
readCommand();
readSensor();
checkFreshness();
if (commandValid && sensorValid) {
  error = reference - estimate;
  correction = Kp * error;
} else {
  correction = 0;
}
logBenchState();
```

Este fragmento es un esquema de organización, no un controlador de vuelo compilable. La referencia concreta c13_mando implementa la normalización y la zona muerta y produce un registro de serie sin salidas de motor. El código c12_imu aporta la lectura y la estimación de banco. El equipo relaciona ambas entradas en su portafolio y conserva las condiciones de validez.

El orden del programa y su duración importan. delay(10) no demuestra un periodo exacto de 10 ms: también consumen tiempo las operaciones de adquisición y de registro. La afirmación del sitio sobre una frecuencia mínima universal que haría caer cualquier dron se retira del material docente. La respuesta de un sistema depende de dinámica, estimación, latencias, actuación y diseño del control, no de una única cifra de bucle.

Laboratorio: mover una cosa cada vez

Clase 13 · Mando y control de banco: interpretar el proyecto de vuelo · 5/6

1. Ejecuta c13_mando y conserva un registro del centro de cada eje.
2. Modifica un extremo de calibración y observa el efecto en la referencia.
3. Comprueba la zona muerta con reposo y movimientos pequeños.
4. Relaciona el eje de mando con la convención del módulo IMU.
5. Resuelve el error y la corrección de tres casos de banco.
6. Invalida una entrada por software y explica la salida prevista.
7. Presenta un diagrama de bloques con unidades, tiempos y estados.

Caso de explicación

Referencia	**Estimación**	**Error**
$10°$	$6°$	$4°$
$0°$	$-3°$	$3°$
$-5°$	$-5°$	$0°$

Los casos numéricos permiten comprobar signos y unidades antes de integrar hardware. En la demostración docente se conserva una entrada y se mueve la otra para que el grupo identifique su efecto. Si ambas cambian simultáneamente, un principiante puede atribuir la corrección a la causa equivocada. Al terminar, el alumno debe poder señalar qué dato cambió y qué parte del programa lo utilizó.

Lo que demuestra el banco y qué amplía la formación

Clase 13 · Mando y control de banco: interpretar el proyecto de vuelo · 6/6

El banco demuestra adquisición, transformación y comparación bajo sus condiciones. Un sistema de vuelo añade muchas otras funciones y validaciones. Para una primera ruta MECRO, el valor está en construir una base comprensible que permita leer después documentación de control y aviónica con mejores preguntas. El alumno conserva su ejemplo como un instrumento de aprendizaje y diagnóstico, con un título que describa la función efectivamente implementada.

Identidad de cada resultado

Resultado	Cómo se presenta
Referencia normalizada	Mando de banco
Ángulo de acelerómetro	Estimación cuasiestática
error × Kp	Acción proporcional
Valores de mezclador impresos	Ejercicio numérico

Investiga por qué una estimación de orientación puede necesitar varias fuentes y cómo un mando humano puede representar ángulo, velocidad angular u otra variable según el modo del sistema. No hace falta implementar esas arquitecturas para reconocer que la palabra “mando” oculta una decisión de interfaz. Describe una de esas posibilidades y qué unidad cambiaría en el error.

Entrega de la clase

Calibración de mando, tres casos resueltos y diagrama de bloques del banco.

Fuentes para ampliar

[MECRO] MECRO. Laboratorio Electrónica con ESP32: veinte proyectos.

[ADC] Espressif Systems. Arduino-ESP32: ADC.

[IMU] Adafruit. MPU6050: biblioteca, cableado y lectura.

[TIME] Arduino. Referencia del lenguaje: millis.

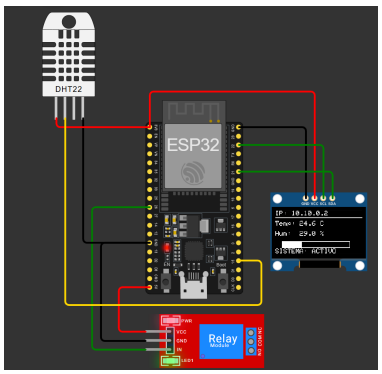
[FBS] Åström y Murray. Feedback Systems: An Introduction for Scientists and Engineers; edición electrónica de los autores.

Qué añadimos al conectar el equipo

Clase 14 · Datos en tierra: Wi-Fi, HTTP y marcas de tiempo · 1/6

Al terminar Publicar o registrar una lectura con su origen temporal y diferenciar conectividad, recepción y dato válido.

La red permite transportar información entre programas. No mejora automáticamente la exactitud del sensor ni garantiza que todos los mensajes lleguen. El proyecto de riego aporta un servidor web; el reloj aporta una referencia de hora. En la ruta de drones reutilizamos esas ideas para observar datos de banco en tierra. La bomba y el agua no son necesarias para aprender la cadena de comunicación.



MECRO, proyecto 19. Esquema original del laboratorio para reconocer la arquitectura en Wokwi. Para montaje físico se aplica la tabla y las correcciones de esta lectura.

El curso diferencia datos locales, mensajes enviados y registros recibidos. Una pantalla remota puede conservar la última respuesta aunque el dispositivo haya dejado de actualizar. Por eso se transportan identificador, tiempo y estado junto al valor. La conexión Wi-Fi es una condición del enlace; la validez de la medición pertenece a otra parte de la aplicación.

La ruta principal enseña una conexión local y una respuesta HTTP sencilla. NTP se utiliza como ampliación guiada de referencia de tiempo. El mouse BLE queda como demostración opcional en hardware, porque el simulador consultado no implementa esa radio. Se conserva el enlace de ampliación sin convertir tres tecnologías de red en un supuesto dominio avanzado durante una sola sesión.

Dirección, solicitud y respuesta

Clase 14 · Datos en tierra: Wi-Fi, HTTP y marcas de tiempo · 2/6

En modo estación, el ESP32 se conecta a una red que le asigna o permite usar una dirección. Un servidor atiende solicitudes sobre una interfaz; un cliente inicia una solicitud. HTTP organiza método, ruta y respuesta. Nuestro recurso /datos devuelve un objeto pequeño con nombres de campo. Una dirección IP mostrada por un simulador no tiene por qué ser accesible desde el navegador de la computadora del alumno.

Términos del ensayo

Término	Función	Ejemplo
SSID	Nombre de red	Red del laboratorio
IP	Dirección del equipo	Observar la asignada
Ruta	Recurso solicitado	/datos
Respuesta	Contenido y estado	JSON de lecturas

Wokwi ofrece conexión saliente a internet mediante su gateway. El acceso entrante a un servidor simulado requiere una configuración de gateway compatible y puede depender del servicio disponible. La práctica gratuita puede observar conexión, datos de serie y consultas salientes; el servidor HTTP local se demuestra con la placa del docente o con el gateway habilitado. El curso no promete abrir la IP simulada desde cualquier celular sin esa condición.

Conectado pero sin dato

El estado Wi-Fi indica conectado, pero /datos muestra una lectura antigua. ¿Qué parte falta revisar?

Cómo lo explicaríamos La adquisición, su marca de tiempo y la generación de la respuesta. Tener enlace no demuestra que exista una medida nueva.

Tiempo relativo y hora UTC

Clase 14 · Datos en tierra: Wi-Fi, HTTP y marcas de tiempo · 3/6

millis() permite ordenar intervalos desde el inicio del equipo. La hora UTC sirve para relacionar registros con una referencia de calendario cuando se obtuvo una sincronización válida. NTP intercambia información temporal a través de la red; no convierte al ESP32 en un reloj atómico ni elimina los errores de latencia y de oscilador. Una lectura previa a la sincronización no se trata como hora correcta por estar escrita en una columna llamada fecha.

Dos referencias de tiempo

Referencia	Útil para	Qué verificar
Tiempo desde inicio	Intervalos locales	Reinicio y retorno
Hora UTC	Relacionar registros	Origen y sincronización
Hora presentada	Lectura humana	Zona y formato

Mismo valor de tiempo

Dos placas informan millis() = 5000. ¿Se demuestra que midieron en el mismo instante global?

Cómo lo explicaríamos No. Cada contador parte del inicio de su equipo. Se necesita una relación entre referencias si se quieren alinear registros.

Para el proyecto final basta conservar una marca relativa coherente por dispositivo. Si el alumno añade UTC, registra también el estado de esa referencia. La precisión temporal necesaria depende de la pregunta: ordenar dos estados de una pantalla no exige lo mismo que relacionar mediciones rápidas de varios dispositivos. La elección se justifica por misión y evidencia.

Una respuesta pequeña que se puede inspeccionar

Clase 14 · Datos en tierra: Wi-Fi, HTTP y marcas de tiempo · 4/6

Esquema del recurso de c14_wifi

```
String payload = "{\"t_ms\":" + String(millis());
payload += ",\"adc\":" + String(analogRead(34)) + "}";
server.send(200, "application/json", payload);
```

La versión completa adquiere el ADC en una agenda y atiende solicitudes en otra. Para hardware se configuran las credenciales de la red de laboratorio; las del simulador no corresponden automáticamente al aula física. Los ejemplos se comparten con valores de demostración, sin guardar contraseñas personales en la entrega pública. El docente muestra dónde se configura la conexión y conserva la práctica en un entorno local.

La demostración puede ampliar el objeto con seq, valid y sample_ms. Esos campos permiten reconocer repetición, antigüedad y estado. El tiempo de la respuesta y el de la adquisición no son necesariamente iguales. El alumno explica cuál informa cada campo y evita actualizar sample_ms simplemente porque alguien pidió la página.

Laboratorio de recepción y antigüedad

Clase 14 · Datos en tierra: Wi-Fi, HTTP y marcas de tiempo · 5/6

1. Abre el proyecto 19 como referencia y sustituye la función de riego por observación de datos de banco.
2. Identifica qué programa actúa como cliente y cuál como servidor.
3. Ejecuta c14_wifi y observa conexión, marca local y lectura en serie.
4. En hardware o gateway habilitado, consulta /datos y compara respuesta y adquisición.
5. Estudia una pérdida de enlace o un valor que deja de actualizarse.
6. Añade estado y marca de adquisición al formato de un mensaje.
7. En la ampliación NTP, diferencia tiempo previo y posterior a una sincronización verificada.

Datos mínimos para revisar un mensaje

Campo	Qué aporta
equipo / seq	Origen y orden
sample_ms	Instante de adquisición
valor / unidad	Significado físico
valid	Estado del dato

Si un alumno no dispone de un servidor accesible en el simulador, realiza la misma revisión sobre el registro local y sobre respuestas identificadas de la demostración docente. No se exige contratar un servicio adicional para demostrar comprensión del formato y de la antigüedad. La limitación de acceso se registra como condición del entorno, separada de un error del circuito.

La extensión BLE y el siguiente nivel

Clase 14 · Datos en tierra: Wi-Fi, HTTP y marcas de tiempo · 6/6

El mouse aéreo reutiliza velocidad angular, zona muerta e interfaz HID sobre BLE en hardware compatible. Es una ampliación útil, pero no reemplaza el estudio de ADC, medida y control. LoRa, registro MicroSD, MQTT, OTA y organización avanzada de tareas pertenecen a otra ruta de formación. La presente sesión ofrece un punto de entrada a datos en tierra y deja una base de registro útil para ese siguiente nivel.

Mensaje y muestra

Se reciben dos respuestas que contienen la misma secuencia y el mismo instante de adquisición. ¿Hay dos muestras nuevas?

Cómo lo explicaríamos No necesariamente. Pueden ser dos transportes de una misma lectura. El identificador y el instante permiten reconocerlo.

Investiga qué puede aportar NTP al ordenar datos y qué no demuestra acerca de la precisión de un sensor. Después compara BLE HID y HTTP por su función de aplicación: uno puede presentar un dispositivo de entrada y el otro intercambiar recursos. El estudiante debe poder elegir una interfaz por lo que necesita transportar, no solo por la presencia de radio en la placa.

Entrega de la clase

Formato de datos, prueba de adquisición y transporte, y nota del entorno utilizado.

Fuentes para ampliar

[MECRO] MECRO. Laboratorio Electrónica con ESP32: veinte proyectos.

[NET] Espressif Systems. Arduino-ESP32: Wi-Fi.

[WIFI] Wokwi. ESP32 WiFi: acceso entrante y saliente.

[WOKWI] Wokwi. ESP32 Simulation: periféricos y límites.

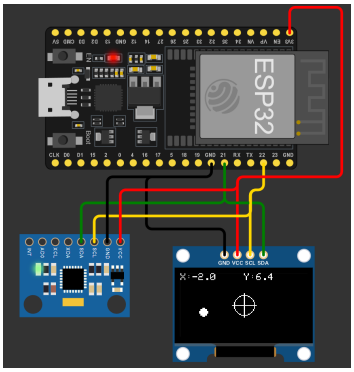
[TIME] Arduino. Referencia del lenguaje: millis.

El proyecto empieza en las clases anteriores

Clase 15 · Proyecto integrador: una estación de pruebas para UAV · 1/6

Al terminar Integrar módulos ya estudiados en un sistema de banco con requisitos y pruebas trazables.

La integración recoge un portafolio que ya creció durante el curso: circuito y medida, entrada digital, agenda, conversión, señal de mando, sensor e interpretación. No se empieza comprando todas las piezas en la clase 15. Cada equipo elige un propósito y conserva el perfil de hardware disponible. El resultado debe poder explicarse con un esquema y comprobarse con un conjunto pequeño de pruebas.



MECRO, proyecto 16. Esquema original del laboratorio para reconocer la arquitectura en Wokwi. Para montaje físico se aplica la tabla y las correcciones de esta lectura.

La opción principal es una estación de IMU y estado: adquiere datos, conserva tiempo y validez, presenta una salida y genera un registro de serie. La ampliación añade un mando analógico o una interfaz de consulta. Como alternativa, un equipo con motor, driver y encoder puede integrar el banco de velocidad con los módulos ya comprobados. La selección depende de componentes y objetivos; no cambia el nivel de exigencia sobre explicación y evidencia.

El curso se puede seguir con simulación y bancos compartidos. El kit facilita el trabajo personal y se entrega a quienes lo soliciten con inventario confirmado. Esa condición comercial no obliga a afirmar que todos los asistentes construyeron físicamente todas las variantes. El expediente identifica el modo real de cada prueba.

Requisitos que permiten ensayar

Clase 15 · Proyecto integrador: una estación de pruebas para UAV · 2/6

Proyecto principal de estación

Requisito	Prueba	Evidencia
Identificar sensor	Inicio con y sin módulo	Estado registrado
Adquirir periódicamente	Observar marcas de tiempo	CSV
Separar dato y validez	Vencer una lectura	Estado de fallo
Presentar estado	Ver salida y registro	Coincidencia explicada
Cambio propio	Comparar dos versiones	Código y conclusión

Un requisito como “que funcione bien” no define una comprobación. Una condición observable podría ser “si no existe una muestra reciente, el registro indica dato no válido”. Para probarla se define qué antigüedad acepta el caso, cómo se inyecta la ausencia y dónde se observa el cambio. El valor de 250 ms del ejemplo es una condición didáctica del banco, no un tiempo universal de sistemas UAV.

Requisito ambiguo

El equipo escribe “detectar cualquier fallo”. ¿Cómo lo convierte en una prueba de esta etapa?

Cómo lo explicaríamos Elige una condición concreta: ausencia al inicializar, dirección sin respuesta, lectura vencida o rango no plausible. Describe el método y la evidencia sin prometer diagnóstico de toda causa posible.

Módulos y responsabilidades

Clase 15 · Proyecto integrador: una estación de pruebas para UAV · 3/6

El código se organiza en adquisición, procesamiento, estado y presentación. Un dato aceptado lleva su instante; la tarea de pantalla no cambia ese origen. La tabla de pines corresponde a la versión final y reemplaza las conexiones de ejercicios anteriores. El semáforo, la OLED y la IMU no se acumulan sobre los mismos GPIO sin revisar su función y sus conflictos.

Perfil del ejemplo integrado

Elemento	Conexión de referencia	Función
IMU I2C	SDA21 / SCL22	Adquisición
Entrada analógica	GPIO34 / ADC1	Lectura de mando
LED con resistor	GPIO23	Estado del dato
Puerto serie	USB-UART de la placa	Registro

La variante de motor conserva, además, el eje y las cuentas por vuelta del encoder, las alimentaciones del driver y los límites de mando. El controlador solo se evalúa después de comprobar medida y sentido de acción. Las curvas del modelo de la clase 11 ayudan a formular la prueba, pero sus ganancias no se trasladan como una promesa de respuesta del hardware.

La base de cámara y el ascensor quedan como ampliaciones de estados y posicionamiento. Sus esquemas publicados requieren unificar motor y driver con el código antes de construir. No se mezclan A4988 de STEP/DIR y una secuencia de cuatro salidas para otro tipo de motor como si fueran la misma interfaz.

Lectura del ejemplo integrado

Clase 15 · Proyecto integrador: una estación de pruebas para UAV · 4/6

Lógica principal de c15_integrador

```
bool valid = ready && haveSample
  && uint32_t(now-lastValid) <= 250;
state = valid ? State::READY : State::FAULT;
digitalWrite(LED, valid);
// El archivo completo conserva INICIO antes de la primera muestra.
// El registro incluye secuencia, tiempo, estado y unidades.
```

El ejemplo comprueba respuesta de bus antes de actualizar el registro. Esta prueba aporta presencia de comunicación en esa operación; no demuestra integridad absoluta de todas las medidas. La plausibilidad numérica y la antigüedad son criterios separados. La estimación angular conserva las hipótesis de la práctica cuasiestática. El alumno debe poder explicar exactamente qué significa data_valid en su propia versión.

El botón queda reservado para una ampliación individual: reconocer un aviso, registrar un marcador o cambiar una presentación. No se da crédito por añadir una línea cuyo comportamiento nadie puede explicar. La modificación se describe como requisito y se prueba con el mismo cuidado que la versión de referencia.

Una marca temporal falsa

El equipo actualiza lastValid cada vez que imprime, aunque la lectura haya fallado. ¿Qué ocurre con el criterio de antigüedad?

Cómo lo explicaríamos Deja de medir edad de la muestra y pasa a medir edad de la impresión. La aplicación puede seguir declarando reciente un dato que ya no lo es.

Laboratorio de integración por hitos

Clase 15 · Proyecto integrador: una estación de pruebas para UAV · 5/6

1. Elige estación de sensores o banco de velocidad según recursos confirmados.
2. Congela una tabla de pines y dibuja energía, señal y referencia.
3. Ejecuta cada módulo por separado y conserva su resultado.
4. Integra dos módulos y comprueba primero una condición normal.
5. Añade el estado de dato y una prueba de ausencia o vencimiento.
6. Introduce una modificación propia con un requisito observable.
7. Prepara el expediente y la demostración del taller final.

Hitos de la entrega

Hito	Condición de revisión
A: subsistemas	Cada uno explica su dato
B: integración	Referencias coherentes
C: diagnóstico	Fallo tratado de forma explícita
D: modificación	Comportamiento propio ensayado

Si aparece un fallo durante integración, se vuelve al último hito comprobado y se cambia una condición. Conservar una versión funcional permite investigar sin borrar la referencia. Un registro de cambio pequeño es suficiente: qué se modificó, qué se esperaba y qué se observó. La documentación acompaña la construcción en lugar de inventarse al final.

Preparación de la defensa

Clase 15 · Proyecto integrador: una estación de pruebas para UAV · 6/6

La demostración debe comenzar por el propósito y el objeto, no por recorrer todas las líneas de código. El equipo presenta una entrada, una transformación y una salida; muestra una condición normal y una condición que cambia el estado; después explica una decisión y su evidencia. Cada integrante responde una pregunta individual sobre su parte. Esa defensa permite distinguir comprensión de reproducción mecánica.

Dos modos de evidencia

Un equipo aporta simulación y otro montaje real. ¿Qué se compara en común y qué se identifica por separado?

Cómo lo explicaríamos Se comparan requisitos, coherencia del código, interpretación y análisis. Se identifica por separado qué prueba física realizó cada equipo y qué condiciones reales pudo observar.

Para investigar, selecciona una ampliación: una pantalla, un registro persistente, una comunicación o un mecanismo. Explica qué interfaz y qué comprobaciones nuevas exige. La siguiente versión se propone con un motivo técnico vinculado al propósito, sin convertir una lista de componentes más larga en una mejora automática.

Entrega de la clase

Primera versión integrada, mapa de pines, cuatro hitos y plan de demostración.

Fuentes para ampliar

[MECRO] MECRO. Laboratorio Electrónica con ESP32: veinte proyectos.

[GPIO] Espressif Systems. GPIO y restricciones de ESP32 clásico.

[WIRE] Espressif Systems. Arduino-ESP32: I2C.

[FBS] Åström y Murray. Feedback Systems: An Introduction for Scientists and Engineers; edición electrónica de los autores.

[WOKWI] Wokwi. ESP32 Simulation: periféricos y límites.

Qué se demuestra al terminar

Clase 16 · Taller final: pruebas, diagnóstico y clase maestra de cierre · 1/6

Al terminar Demostrar el sistema, explicar una condición de fallo y defender una mejora con evidencia.

La demostración final reúne construcción, programación y explicación. Se presenta una configuración identificada con un objetivo y un método de prueba. Una aplicación que solo muestra números bonitos puede ocultar referencias equivocadas; una que documenta su estado y sus datos permite investigar lo ocurrido. El taller evalúa ese criterio con el equipo delante.

Paquete de entrega

Elemento	Qué permite revisar
Esquema vigente	Energía, señal y referencia
Código y entorno	Reproducir la construcción
Registro	Tiempo, unidad y estado
Pruebas	Condiciones y observaciones
Defensa	Decisiones del estudiante

El proyecto principal se demuestra en banco; no se plantea como firmware de vuelo. Un equipo que trabajó por simulación identifica sus resultados como tales y participa en las comprobaciones compartidas disponibles. La evaluación conserva esa diferencia para que un certificado del curso no prometa una experiencia física que no se documentó.

Antes de energizar se revisa que el montaje corresponda al esquema de esa versión. Las modificaciones realizadas después de una prueba pueden afectar la referencia, los pines o el estado inicial. El expediente no necesita ser complicado: debe permitir reconocer qué sistema se examinó y qué condiciones se conservaron.

Agenda del taller y roles

Clase 16 · Taller final: pruebas, diagnóstico y clase maestra de cierre · 2/6

Taller presencial 2: cuatro horas

Minutos	Actividad	Producto
0-25	Revisión de versiones	Lista de equipos
25-80	Prueba normal y registro	Evidencia de funcionamiento
80-140	Pruebas de fallo	Diagnóstico observado
140-195	Demostraciones y defensa	Explicación individual
195-220	Correcciones pequeñas	Revisión de cambio
220-240	Cierre y siguiente ruta	Informe final

Los roles de montaje, software, medición y registro rotan a lo largo del curso. En el cierre, cada integrante explica al menos un dato y una decisión. El docente puede pedir una modificación pequeña y controlada o presentar un caso nuevo del registro. No se trata de memorizar el programa del equipo, sino de poder usar sus conceptos para responder una pregunta cercana.

Los bancos compartidos se programan según recursos reales. Si hay más equipos que puestos, unos ejecutan la prueba mientras otros revisan registros y preparan la defensa. El plan docente conserva quién realizó cada operación y quién participó como observador. Esa organización mantiene una evaluación justa para alumnos con y sin kit propio.

Una matriz de pruebas que produce conclusiones

Clase 16 · Taller final: pruebas, diagnóstico y clase maestra de cierre · 3/6

Casos del proyecto principal

Caso	Qué se observa	Resultado esperado
Inicio normal	Sensor y registro	Dato nuevo identificado
Sensor ausente al inicio	Estado de inicialización	Fallo explícito
Muestra vencida	Edad del último dato	Dejar de declararlo reciente
Entrada al extremo	Mando y límites	Rango definido
Registro repetido	Secuencia e instante	Reconocer duplicación

Cada caso necesita un método. La ausencia inicial puede ensayarse con una configuración preparada por el docente. La muestra vencida puede inyectarse por software sin intervenir el cableado durante operación. Un resultado de inyección se identifica como tal y permite comprobar la lógica específica. El método no demuestra automáticamente todas las causas físicas de fallo del sensor.

Fallo observado y causa propuesta

La IMU no aparece en el inicio. ¿Se puede concluir inmediatamente “el sensor se quemó”?

Cómo lo explicaríamos No. Se revisan alimentación, referencia, conexión, dirección y configuración. El resultado demuestra que no se obtuvo la condición de inicialización; la causa requiere más evidencia.

La matriz convierte una afirmación amplia de funcionamiento en condiciones verificables. Si el resultado no coincide, se conserva la observación antes de cambiar código o cables. La comparación entre lo esperado y lo obtenido guía el diagnóstico y deja una evidencia que otro equipo puede entender.

Leer el registro de una prueba

Clase 16 · Taller final: pruebas, diagnóstico y clase maestra de cierre · 4/6

Ejemplo sintético de formato CSV

```
seq,t_ms,state,data_valid,roll_deg,pitch_deg
10,1000,1,1,2.1,-0.3
11,1100,1,1,2.2,-0.3
12,1400,2,0,2.2,-0.3
```

La última fila conserva un valor numérico, pero lo marca como no válido en esa condición. Si alguien grafica solo roll_deg podría interpretar que nada cambió. El estado y el tiempo cuentan otra parte de la historia. Los datos de este ejemplo son sintéticos y se usan para enseñar lectura de registros; los del equipo se sustituyen por su ejecución identificada.

En el banco de velocidad se revisan también referencia, RPM, mando, ventana y límites. Una respuesta lenta puede relacionarse con la planta, con la medida o con el controlador. El informe separa observación e hipótesis y propone qué nueva prueba ayudaría a distinguir causas. No se cambia todo a la vez hasta lograr una curva estéticamente agradable.

Una mejora con una pregunta

El equipo propone “poner más filtros” porque vio ruido. ¿Qué debe comprobar?

Cómo lo explicaríamos Qué variación quiere reducir, con qué referencia la distingue y qué retardo o pérdida de detalle introduce la propuesta. La mejora se evalúa por función y evidencia.

Evaluación y defensa individual

Clase 16 · Taller final: pruebas, diagnóstico y clase maestra de cierre · 5/6

Evaluación del curso

Componente	Peso	Qué se valora
Portafolio individual	30 %	Predicción y explicación
Prácticas y registros	25 %	Método y evidencia
Proyecto integrado	35 %	Requisitos y pruebas
Defensa individual	10 %	Comprensión y transferencia

1. Presenta propósito y configuración en un minuto.
2. Señala energía, una entrada, una transformación y una salida.
3. Explica una fila del registro con unidades y referencia temporal.
4. Muestra una prueba de fallo y su alcance.
5. Defiende una modificación propia con su comparación.
6. Declara una limitación y la prueba que harías después.

El docente distingue entre un cálculo correcto, un programa que compila, una simulación ejecutada y una comprobación física. Son evidencias complementarias. El alumnado recibe retroalimentación sobre qué logró demostrar y qué puede mejorar. Esa información sirve para elegir la siguiente ruta de formación y para presentar el proyecto de manera profesional.

Lo que sigue después de este curso

Clase 16 · Taller final: pruebas, diagnóstico y clase maestra de cierre · 6/6

La ruta siguiente puede profundizar en aviónica y telemetría, registro MicroSD, comunicaciones dedicadas, estimación de orientación o diseño de control. Otra ruta puede desarrollar mecanismos y robótica. La base obtenida permite elegir con criterio: el estudiante ya conoce entradas y salidas, tiempos, medida, potencia, referencias y pruebas. No necesita afirmar dominio de todos los subsistemas de un UAV para presentar un buen proyecto introductorio.

Interés del estudiante

Interés	Base del curso	Siguiente profundización
Sensores de UAV	Adquisición y unidades	Calibración y estimación
Mecanismos	Señal, potencia y medida	Control y diseño mecánico
Datos en tierra	Registros y HTTP	Telemetría y almacenamiento

La entrega final incluye una propuesta de siguiente versión con una sola mejora prioritaria. Debe nombrar el problema observado, el cambio propuesto y una comprobación. Un proyecto pequeño puede tener gran valor formativo si su constructor entiende lo que mide, lo que decide y lo que todavía necesita estudiar. Ese es el criterio que MECRO conserva al ampliar su oferta.

Entrega de la clase

Expediente final, matriz de pruebas, registro y defensa individual.

Fuentes para ampliar

[MECRO] MECRO. Laboratorio Electrónica con ESP32: veinte proyectos.

[GPIO] Espressif Systems. GPIO y restricciones de ESP32 clásico.

[WIRE] Espressif Systems. Arduino-ESP32: I2C.

[FBS] Åström y Murray. Feedback Systems: An Introduction for Scientists and Engineers; edición electrónica de los autores.

[WOKWI] Wokwi. ESP32 Simulation: periféricos y límites.